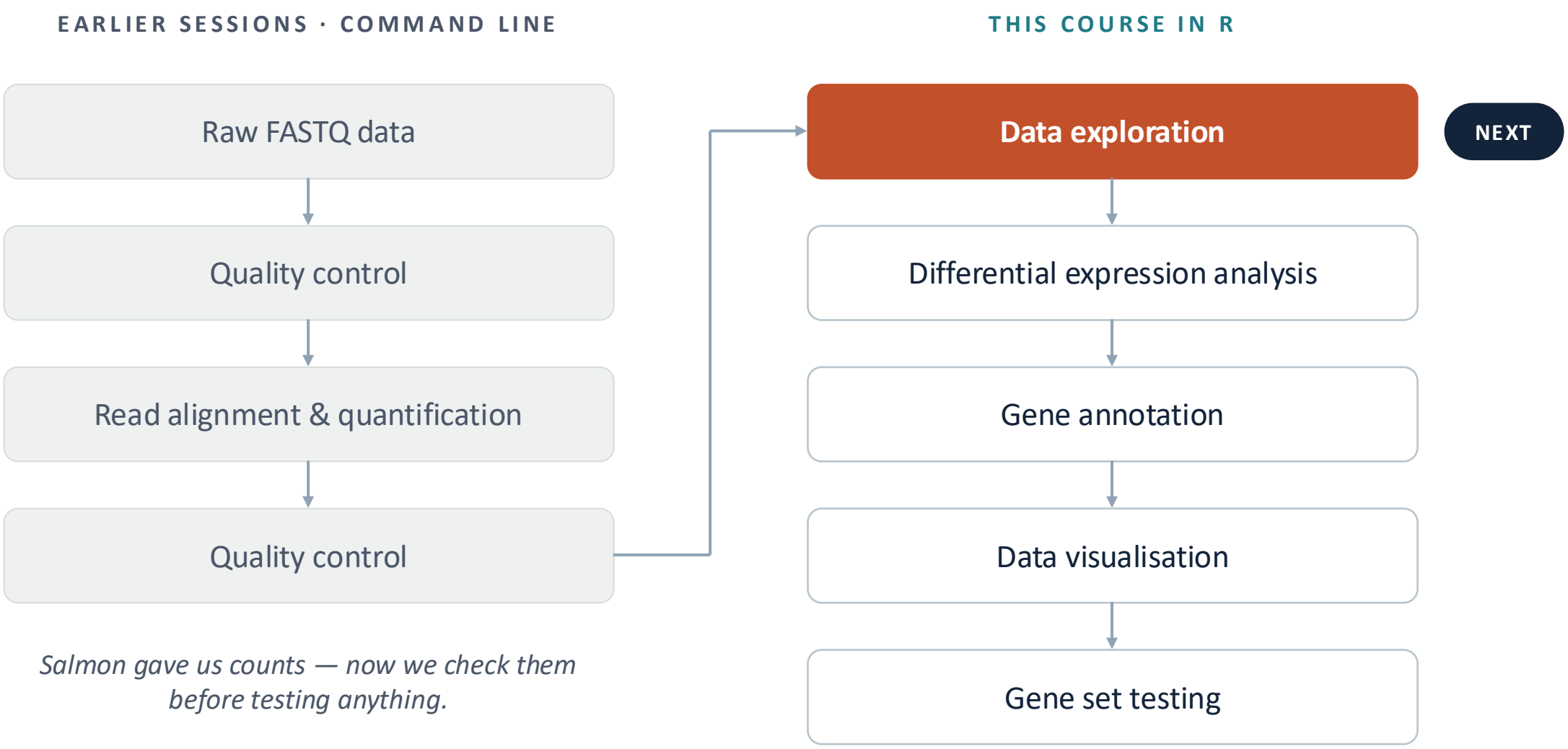
INTRODUCTION TO BULK RNA-SEQ DATA ANALYSIS

RNA-seq Data Analysis Practicals

Systems Biology Part III

Roslin Russell

Where this session fits



Salmon gave us counts — now we check them before testing anything.

Why explore before testing?

Before any statistics, we check that patterns in the raw data match what we know about the experimental design.

1

Batch effects

Samples group by when or how they were processed, not by biology.

2

Outlier samples

One replicate sits far from its group and can distort the analysis.

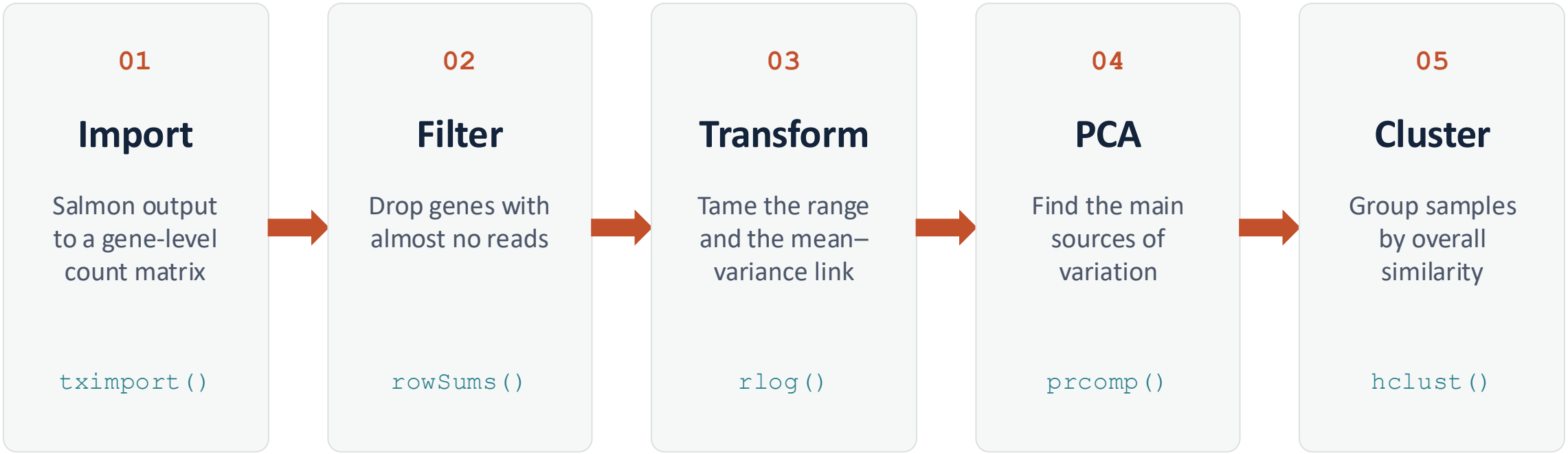
3

Sample swaps

A sample is mislabelled in the metadata and lands in the wrong group.

Our tools: **summary statistics** and **visualisations**

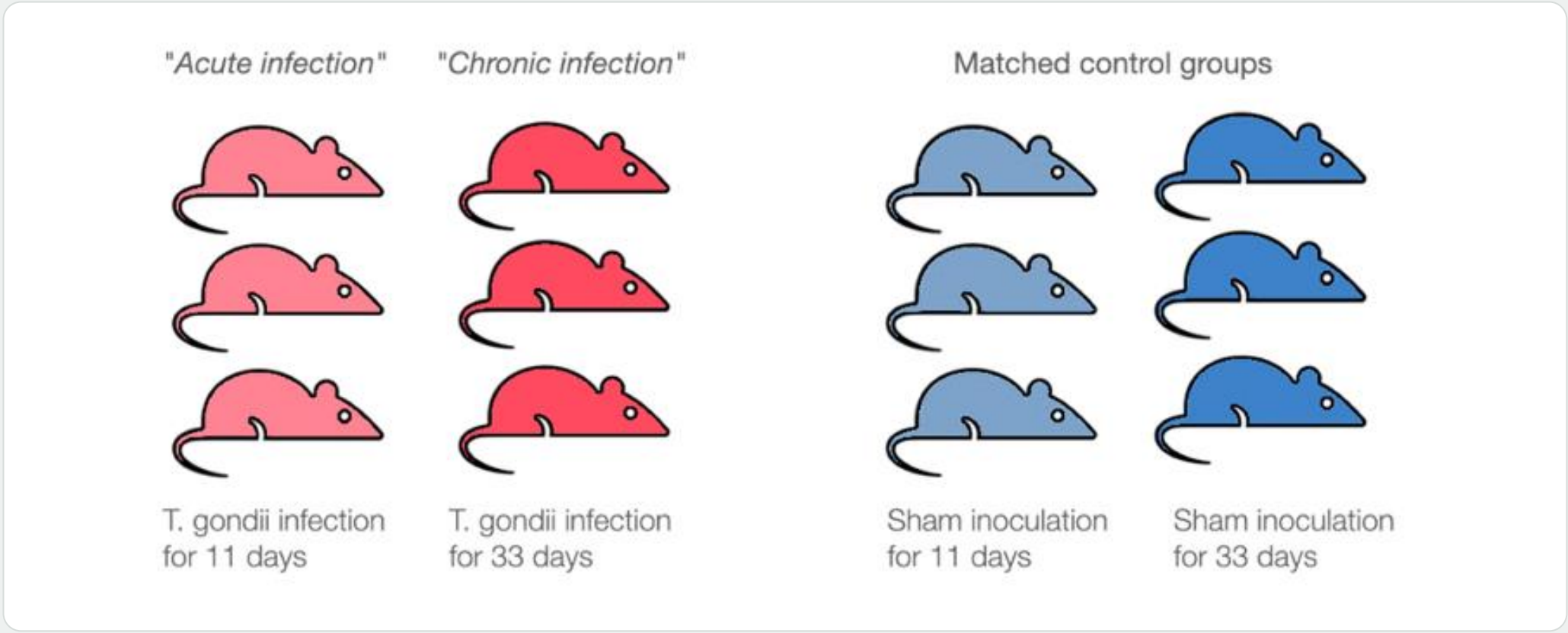
Data Exploration Workflow



Next session: **differential expression** with **DESeq2** on the raw counts.

The dataset: mouse brain and *Toxoplasma gondii*

Gene expression in the brains of mice infected with *T. gondii* oocysts, compared with uninfected controls.



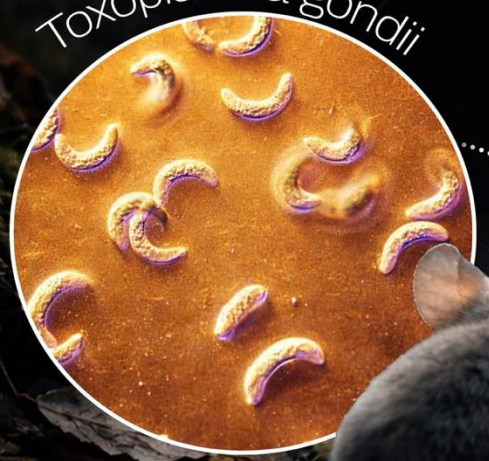
2 × 2 design
Status × Time point, 3 replicates each: 12 samples

Hu et al. (2020)
Frontiers in Microbiology 11: 2529

Raw reads
NCBI SRA project PRJNA483261

Journal Plos One

THE BRAIN MAZE



This parasite can *suppress fear* in animals, causing them to approach predators instead of avoiding them.

The dataset: mouse brain and *Toxoplasma gondii*

Two factors with two levels each:

Status and **Time point** (11 and 33 days post infection).



	11 dpi	33 dpi
Infected	3 replicates	3 replicates
Uninfected	3 replicates	3 replicates

12 samples in total

Importing counts with `tximport`

`quant.sf` × 12

Salmon estimates per transcript, one file per sample

+

`tx2gene.tsv`

Maps 119,414 transcript IDs to gene IDs



`txi$counts`

35,896 genes × 12 samples, summed to gene level

```
files <- file.path("salmon", sampleinfo$SampleName, "quant.sf")
files <- set_names(files, sampleinfo$SampleName)
tx2gene <- read_tsv("references/tx2gene.tsv")
txi <- tximport(files, type = "salmon", tx2gene = tx2gene)
# Salmon counts are estimates, so round for exploration
rawCounts <- round(txi$counts, 0)
```

Filtering out unexpressed genes

35,896

genes in the annotation

20,091

kept: more than 5 reads in total

15,805

removed as likely technical noise

```
keep <- rowSums(rawCounts) > 5
filtCounts <- rawCounts[keep, ]
```

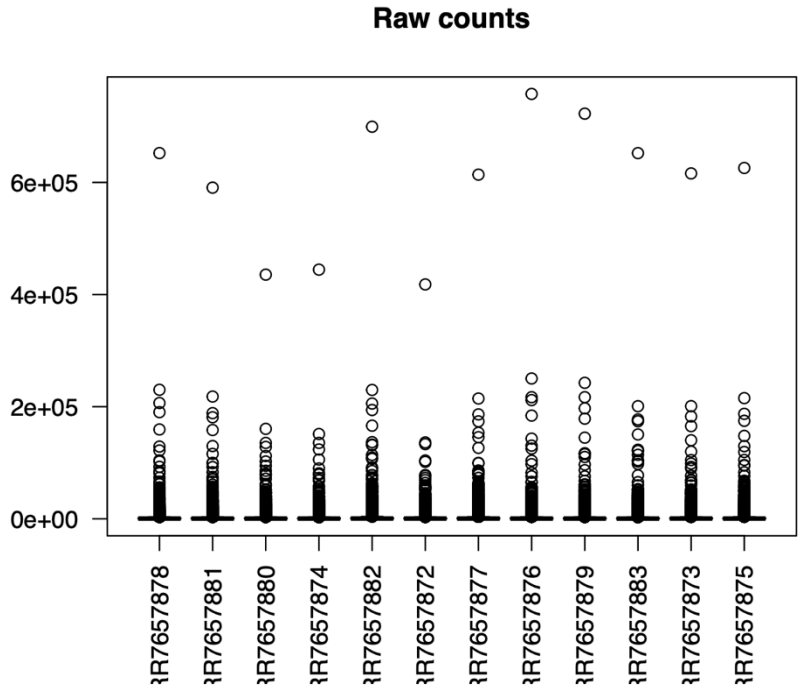
The cut-off is a judgement call: it balances removing noise against keeping biology. It is for **visualisation only** and does not affect the differential expression results.

Why not plot the raw counts?

PROBLEM 1

An enormous range

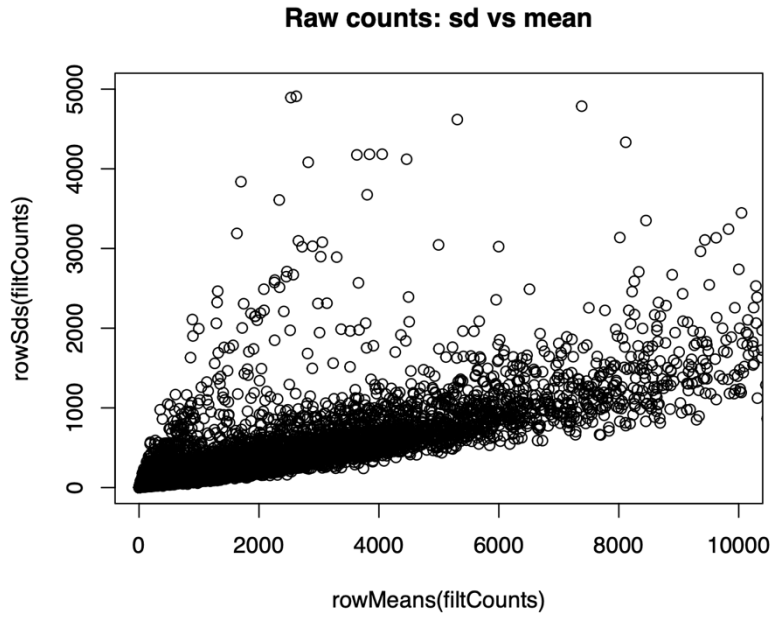
Medians sit around 300–400 reads, but a few genes exceed 750,000 — the boxes collapse to a line.



PROBLEM 2

Variance grows with the mean

Highly expressed genes vary most, so they dominate distances in clustering and PCA.



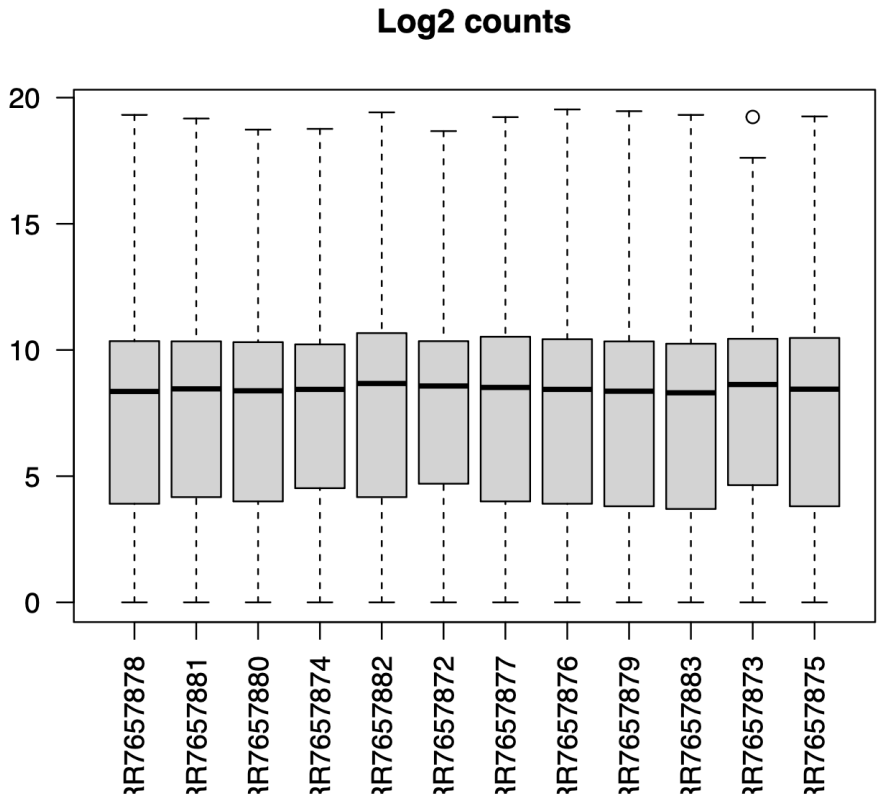
Choosing a transformation

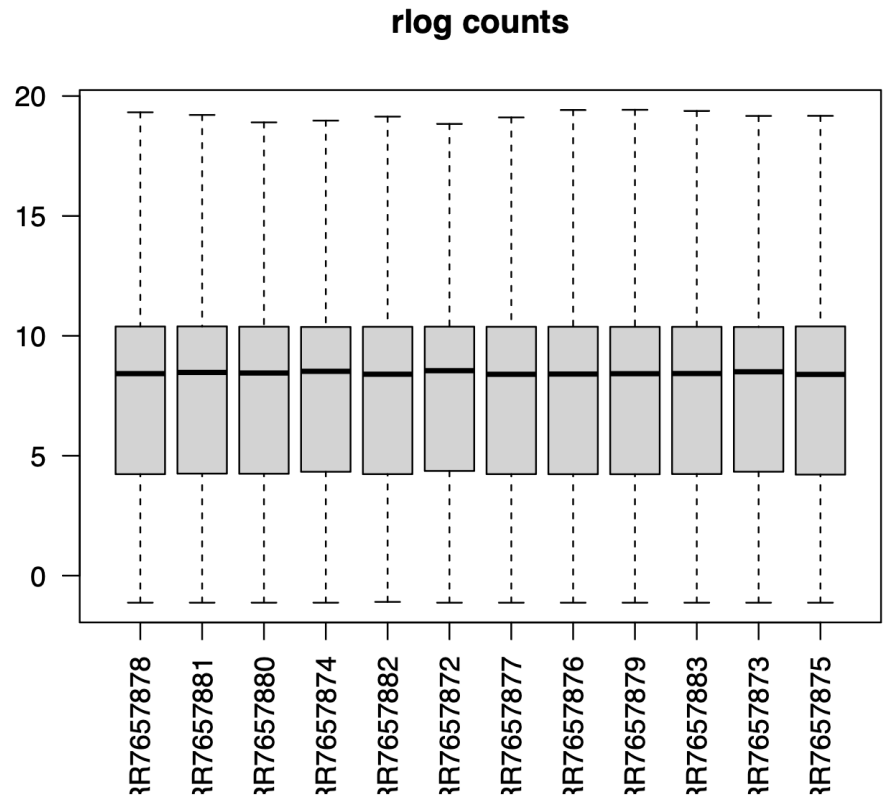
	log2(x + 1)	rlog()	vst()
Compresses the range	Yes	Yes	Yes
Normalises library size	No	Yes	Yes
Stabilises variance	No	Yes	Yes
Speed	Instant	Slow on big data	Fast
Best for	A quick look	Small studies, uneven library sizes	Larger datasets

```
rlogcounts <- rlog(filtCounts)
```

With only 12 samples, **rlog** is affordable — so that is what we use.

log2 versus rlog





`log2(x + 1)`

The range is tamed, but boxes still shift between samples because library sizes differ.

`rlog()`

Normalised for library size: the distributions now line up across all 12 samples.

PCA: does the data reflect the design?

An **unsupervised** view: we don't tell PCA the groups.

In a well-run experiment we expect:

- ✓ Replicates of the same group to sit close together
- ✓ The largest variation to separate treatment groups
- ! Anything else to flag outliers, batch effects or swaps

```
# samples must be rows
pcDat <- prcomp(t(rlogcounts))

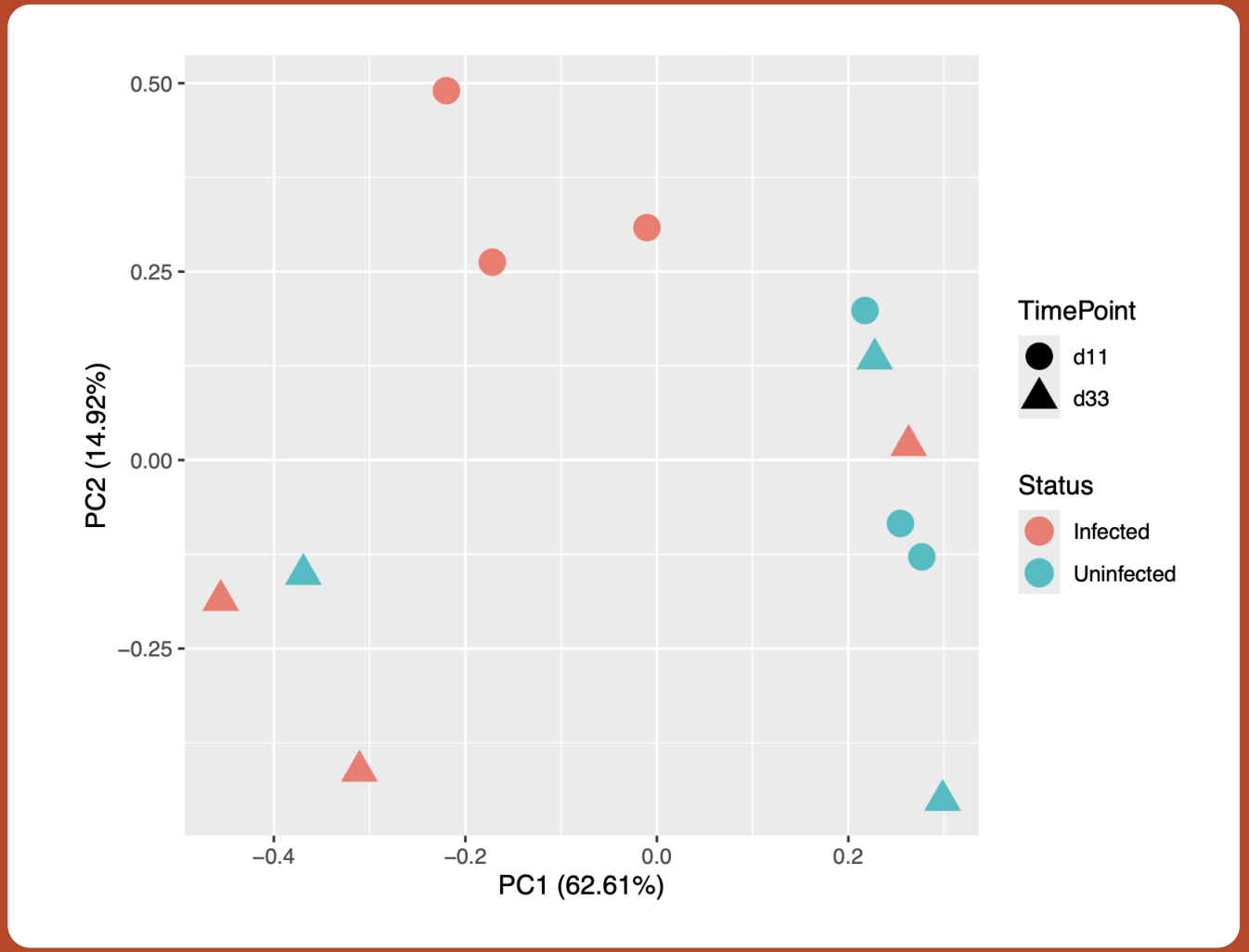
autoplot(pcDat,
  data = sampleinfo,
  colour = "Status",
  shape = "TimePoint",
  size = 5)
```

autoplot() comes from ggfortify, which builds on ggplot2.

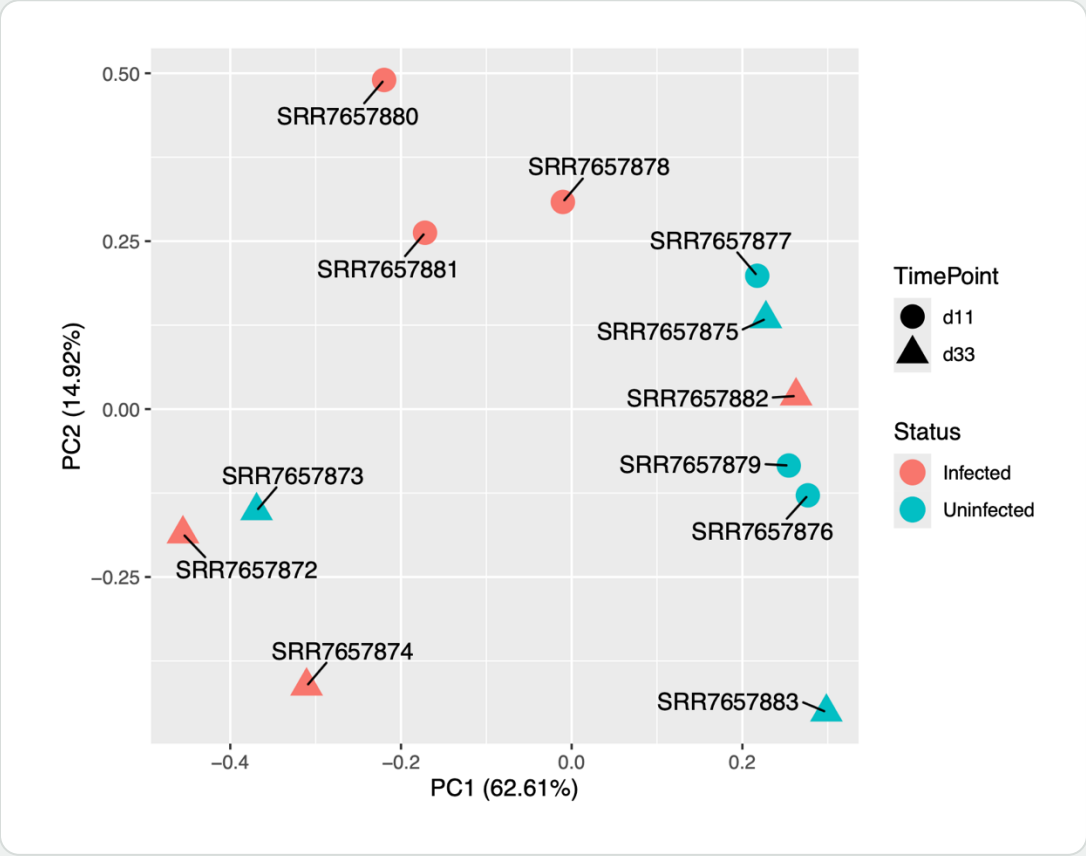
LOOK CLOSELY

Does every sample sit with its group?

Label each point with ggrepel::geom_text_repel() and look for anything out of place.

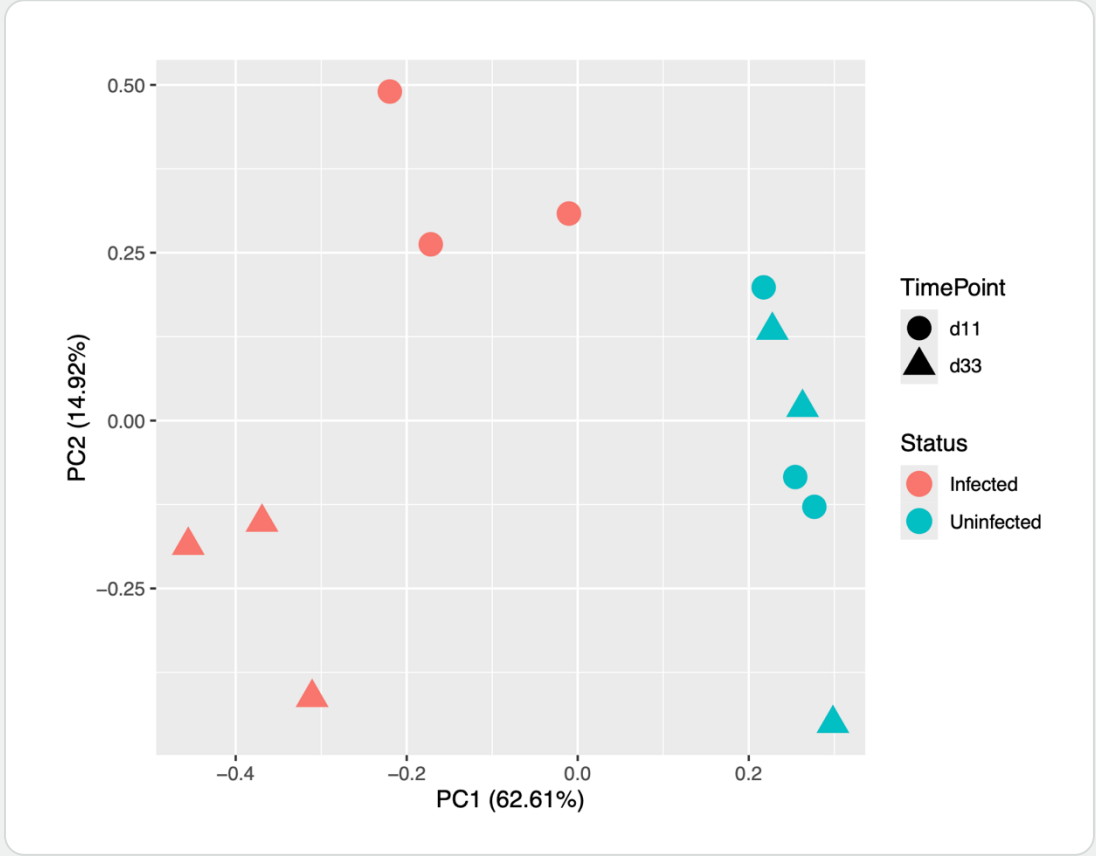


Two samples were swapped in the sample sheet



BEFORE

SRR7657882 is labelled Infected but sits with Uninfected; SRR7657873 is labelled Uninfected but sits with Infected d33.



AFTER

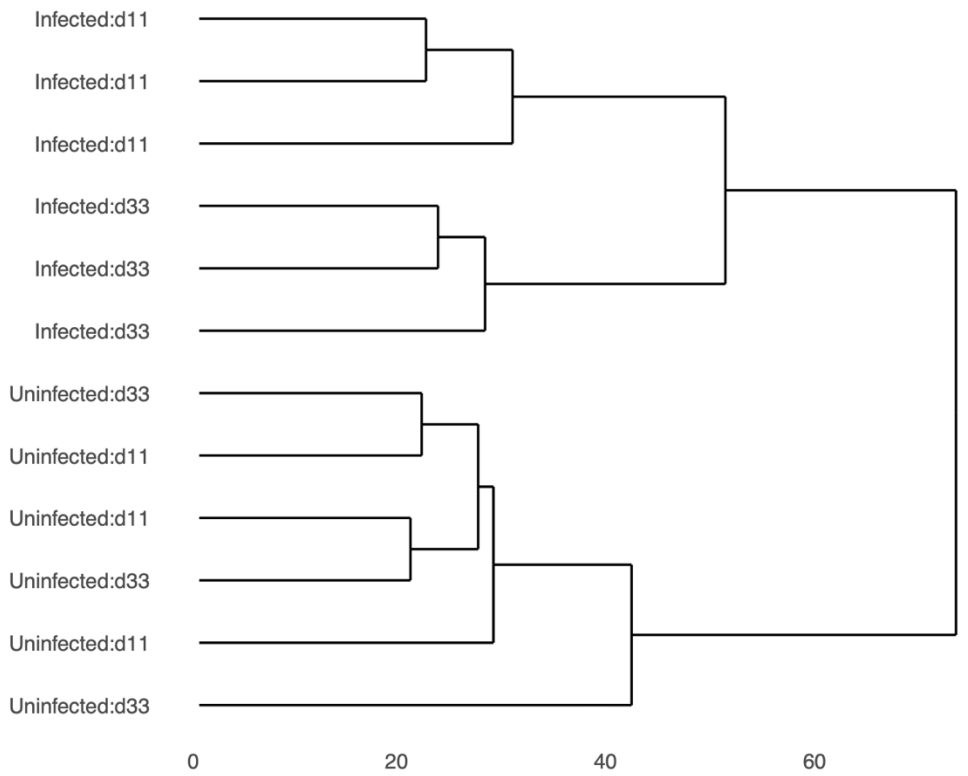
Corrected with mutate() and case_when(): every replicate now sits with its group.

Hierarchical clustering: a second view

```
hclDat <- t(rlogcounts) %>%
  dist(method = "euclidean") %>%
  hclust()

ggdendrogram(hclDat, rotate = TRUE)
```

Tip labels swapped for Status:TimePoint from the corrected sample sheet.



? Do infected and uninfected split at the top?

? Do day 11 and day 33 separate within each status?

? Does the tree agree with the PCA?

Over to you

1. Import the Salmon output and build the count matrix
2. Filter genes and compare raw, log2 and rlog boxplots
3. Run the PCA, label the samples and fix the sample sheet
4. Cluster the samples and compare with the PCA

EXERCISE

Redraw the PCA with PC2 on the x-axis and PC3 on the y-axis.

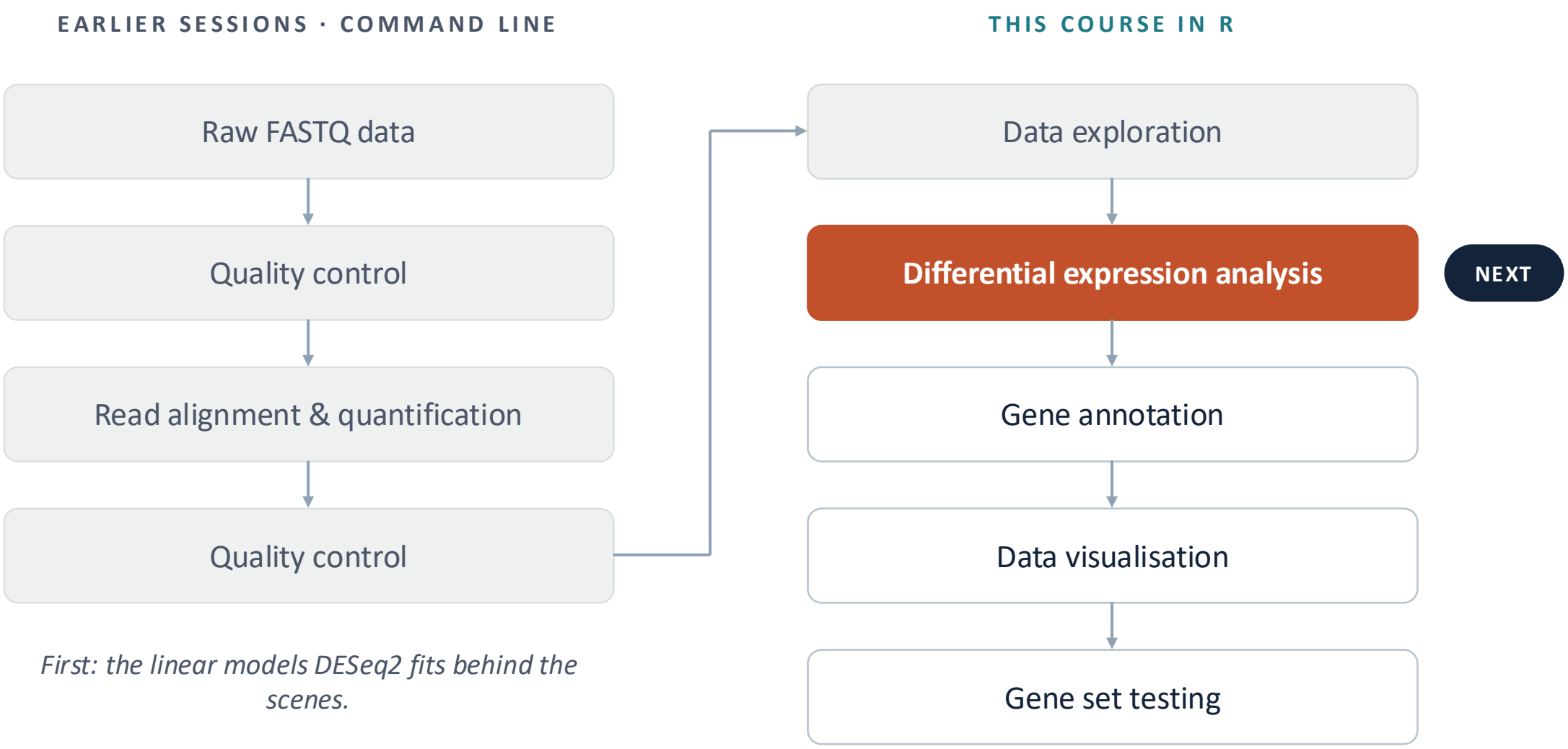
Hint: see `?autoplot.prcomp`

DISCUSS

What do the PCA plots tell us about our samples?

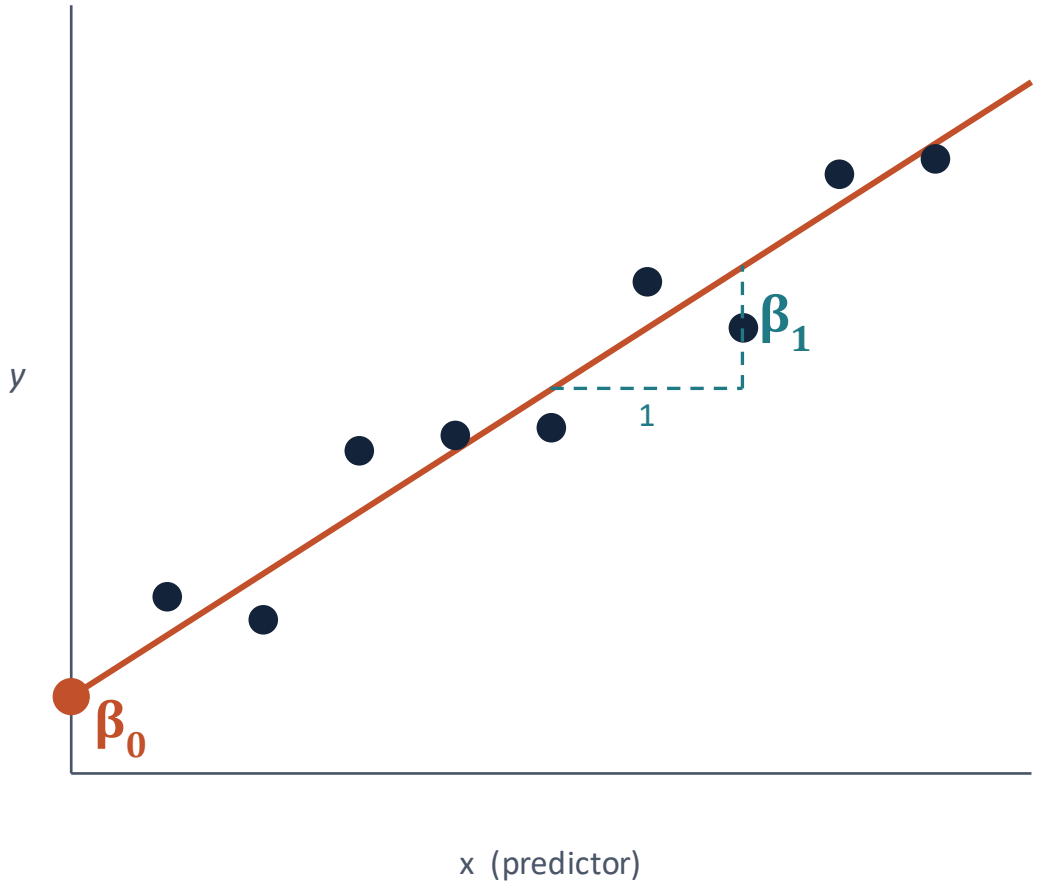
`tximport` · `DESeq2` · `tidyverse` · `ggfortify` · `ggrepel` · `ggdendro`

Next: differential expression



First: the linear models DESeq2 fits behind the scenes.

A model describes how variables relate



$$y = \beta_0 + \beta_1 x$$

y is the outcome; **x** is the predictor that influences it.

β_0 intercept: y when x = 0

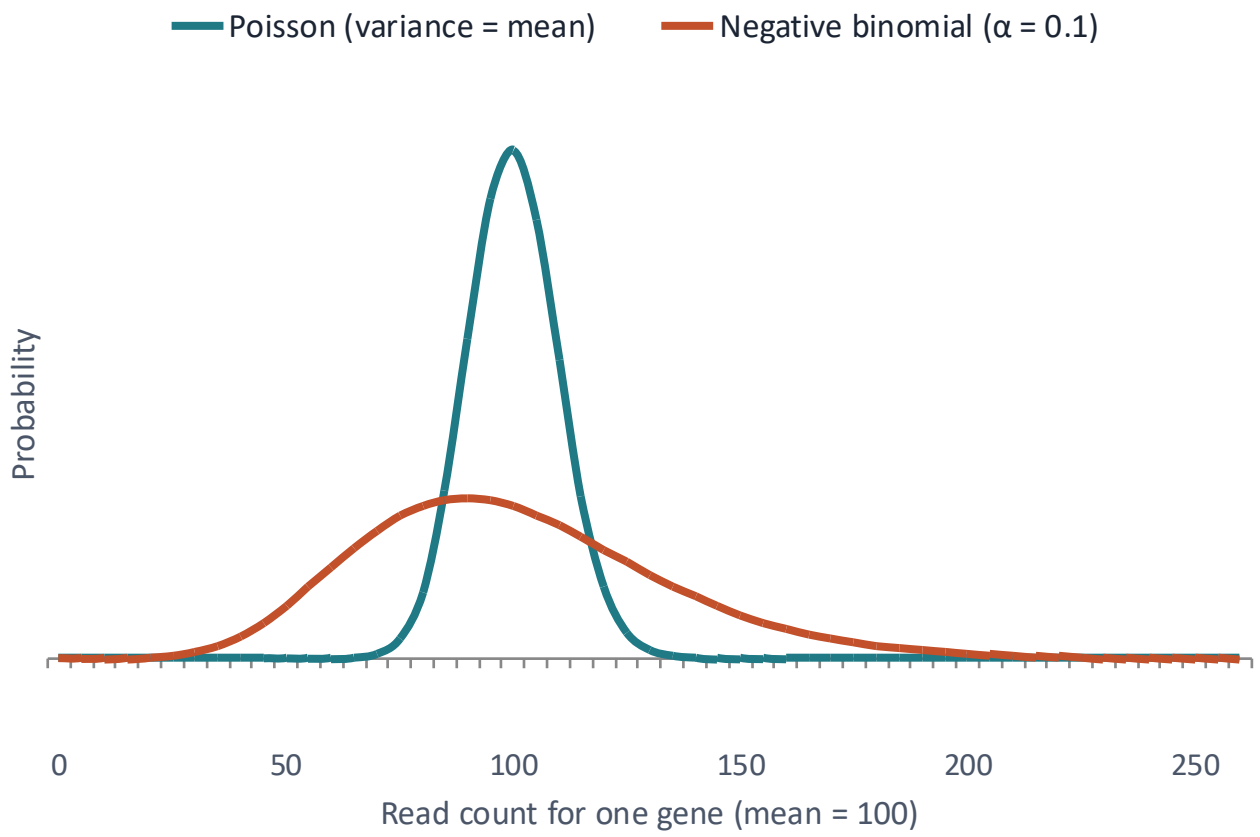
β_1 slope: change in y per unit of x

FOR RNA-SEQ COUNTS

$$\text{counts} \sim \text{Negative Binomial}(\mu, \varphi)$$
$$\log(\mu) = \beta_0 + \beta_1 x$$

Same idea, on the log scale of the mean.

Why a negative binomial?



Same mean, but replicates spread far wider than counting noise alone predicts.

COUNTS

Not normal

Whole numbers, never negative, skewed, many low values

POISSON

Variance = mean

Fits technical replicates: counting noise only

NEGATIVE BINOMIAL

Variance = $\mu + \alpha\mu^2$

Adds biological variation between replicates (overdispersion)

Mean	Poisson SD*	NB SD (α = 0.1)
10	3.2	**4.5**
100	10	**33**

Poisson SD* = √variance e.g. Poisson at mean 10: √10 ≈ 3.2

Groups become indicator variables

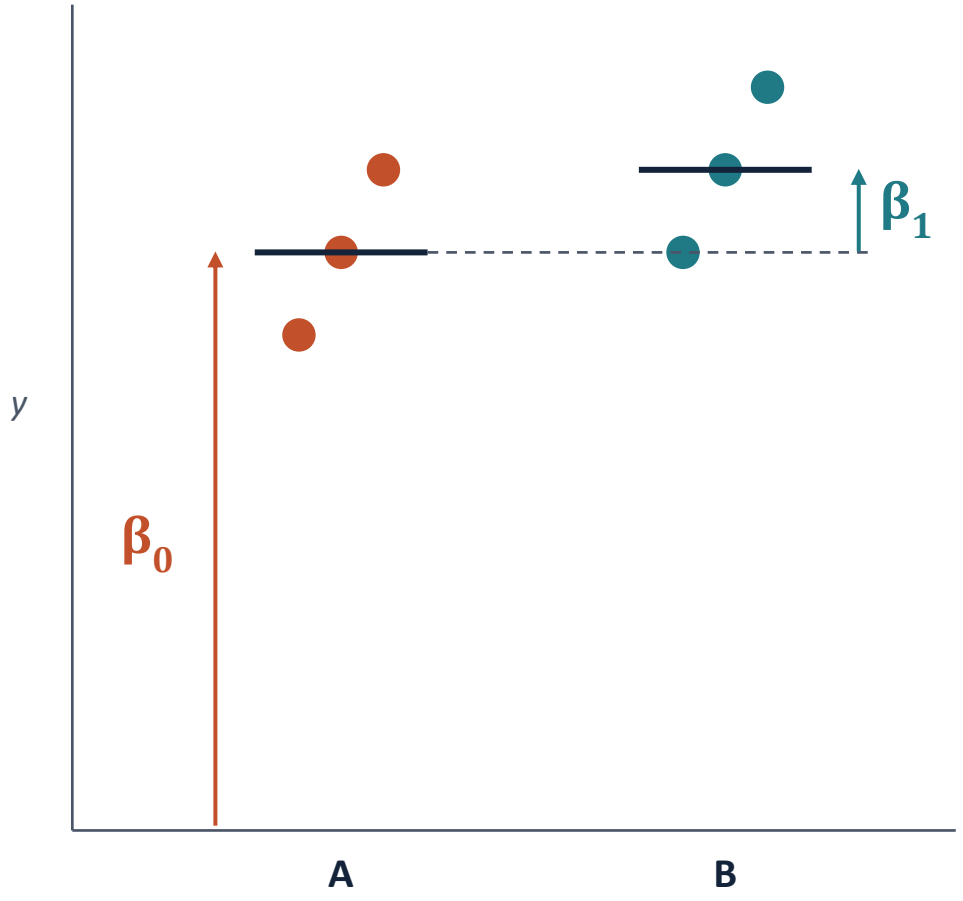
$$y = \beta_0 + \beta_1 x_B$$

β_0 the average of the reference group, A

β_1 the difference between B and A

sample	group	x_B
sample1	A	0
sample2	A	0
sample3	A	0
sample4	B	1
sample5	B	1
sample6	B	1

R builds these 0/1 columns for you — **you just need to know how to read them.**



Testing whether a coefficient is zero

Null hypothesis: the predictor has no effect, so $\beta_1 = 0$

Estimate β_1

Our best guess from the data, e.g. the log2 fold change

Uncertainty $\sigma_{\beta 1}$

The standard error: how precisely we know it

$z = \beta_1 / \sigma_{\beta 1}$

Test statistic: the estimate measured in units of its uncertainty

p-value

How compatible the data are with $\beta_1 = 0$

✓ **Low p-value**

The data are hard to explain if $\beta_1 = 0$, so we reject the null hypothesis.

! **High p-value ≠ no effect**

The data are compatible with $\beta_1 = 0$ — but perhaps also with other values, if the data are noisy or replicates are few.

Formulas and model matrices in R

```
sample_info <- read_tsv("data/samplesheet_corrected.tsv")
status_model <- formula(~ Status)
model.matrix(status_model, data = sample_info)

# make Uninfected the reference level
sample_info$Status <- fct_relevel(sample_info$Status, "Uninfected")
```

DEFAULT: ALPHABETICAL

Status	(Intercept)	StatusUninfected
Infected	1	0
Infected	1	0
Uninfected	1	1
Uninfected	1	1

AFTER fct_relevel()

Status	(Intercept)	StatusInfected
Infected	1	1
Infected	1	1
Uninfected	1	0
Uninfected	1	0

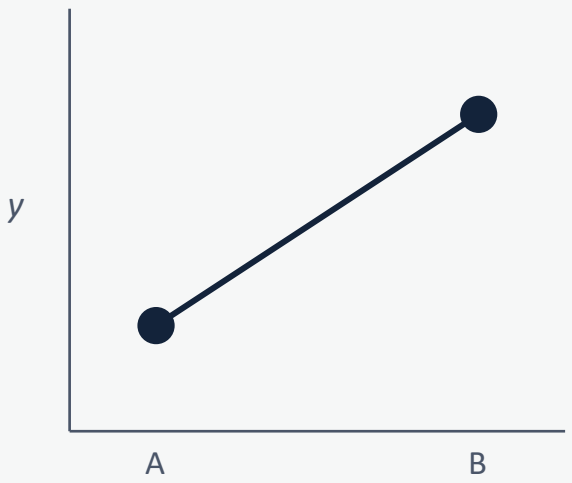
`~ Status` reads as "expression depends on Status".

The column R leaves out is the **reference level** — chosen alphabetically unless you set it.

Three common designs

Single factor

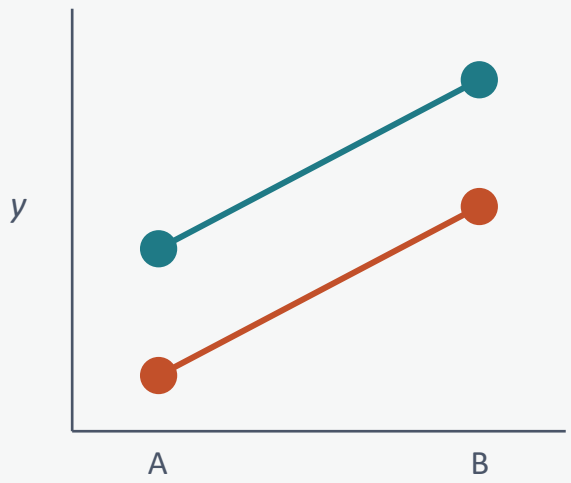
```
~ treatment
```



One coefficient per non-reference level.

Two factors, additive

```
~ genotype + treatment
```

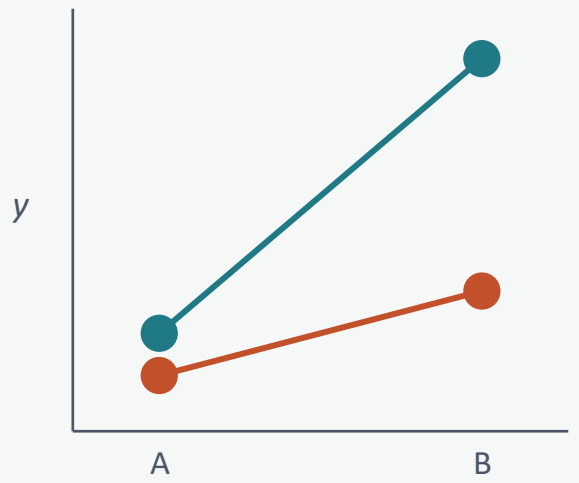


WT MUT

Effects add up: the lines are parallel.

Two factors, interaction

```
~ genotype + treatment
  + genotype:treatment
```



WT MUT

The treatment effect depends on genotype.

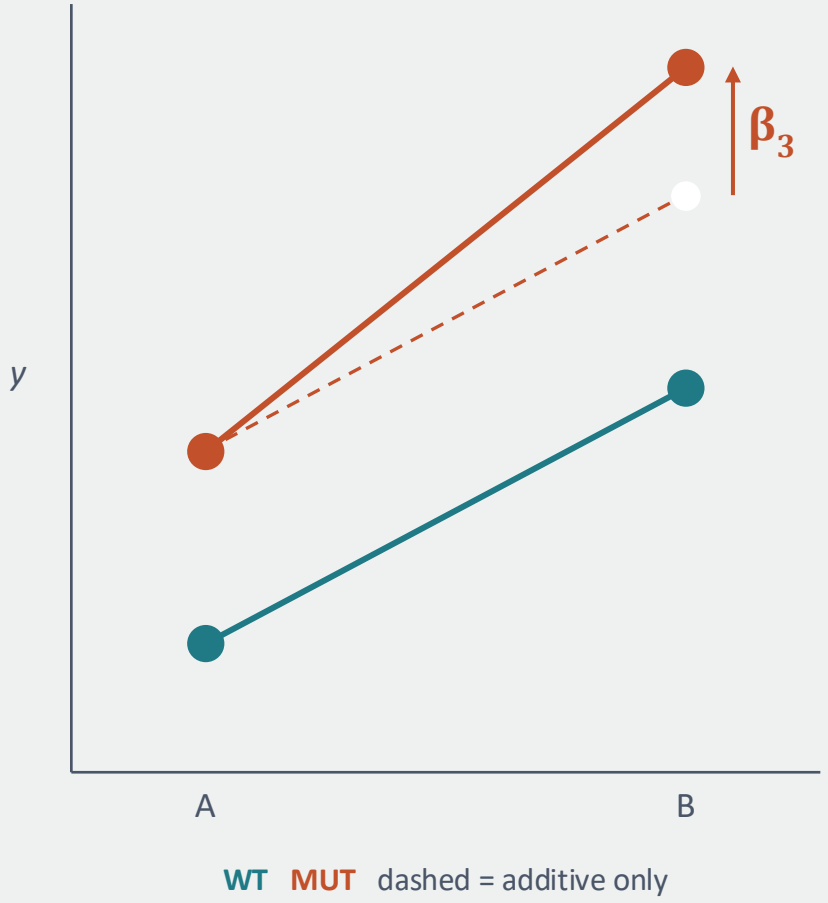
Reading an interaction model

$$y = \beta_0 + \beta_1 \cdot MUT + \beta_2 \cdot B + \beta_3 \cdot MUT\&B$$

Example: $\beta_0 = 1$, $\beta_1 = 1.5$, $\beta_2 = 2$, $\beta_3 = 1$

Group	Terms that apply	Prediction
WT, A (reference)	β_0	1
WT, B	$\beta_0 + \beta_2$	3
MUT, A	$\beta_0 + \beta_1$	2.5
MUT, B	$\beta_0 + \beta_1 + \beta_2 + \beta_3$	5.5

β_3 is the **"difference of differences"**: the extra nudge MUT gets in treatment B beyond what the two main effects predict.



Exercise: spot the interaction

1. Which panels show an interaction between treatment and genotype?
2. In which do the genotypes differ under control conditions?



Specifying the model in DESeq2

```
# sample info lives in colData(dds)
design(dds) <- ~ treatment

# fit a negative binomial model to every gene
dds <- DESeq(dds)

resultsNames(dds)

# choose the comparison you want
results(dds, contrast = list("treatment_B_vs_A"))
```

resultsNames() maps onto the model:

`Intercept`	β_0	Reference level (A)
`treatment_B_vs_A`	β_1	B – A
`treatment_C_vs_A`	β_2	C – A

Comparing two non-reference levels, C vs B, tests $\beta_2 - \beta_1 = 0$:

```
list("treatment_C_vs_A",
     "treatment_B_vs_A")
```

Reading the results table

Column	What it is
`baseMean`	Mean normalised expression across all samples
`log2FoldChange`	The estimated β: log2 of group 1 ÷ group 2
`lfcSE`	Standard error of that estimate (σβ)
`stat`	Wald statistic = log2FoldChange ÷ lfcSE
`pvalue`	Compatibility with log2FoldChange = 0
`padj`	p-value adjusted for multiple testing (FDR)

WHY LOG2?

Up and down changes become symmetric:

4× → **+2**

2× → **+1**

1× → **0**

½× → **−1**

¼× → **−2**

Positive: higher in the first group. Negative: higher in the second.

Asking questions of an interaction model

```
design(dds) <- ~ genotype + treatment + genotype:treatment
```

Null hypothesis	contrast =	Question answered
$\beta_1 = 0$	`list("genotype_MUT_vs_WT")`	Do WT and MUT differ in treatment A?
$\beta_2 = 0$	`list("treatment_B_vs_A")`	Do treatments A and B differ for WT?
$\beta_3 = 0$	`list("genotypeMUT.treatmentB")`	Do the genotypes respond differently to treatment?
$\beta_1 + \beta_3 = 0$	`list(c("genotype_MUT_vs_WT", "genotypeMUT.treatmentB"))`	Do WT and MUT differ in treatment B?

In our Toxoplasma data (reference levels Uninfected and d11), the same model gives `Status_Infected_vs_Uninfected`, `TimePoint_d33_vs_d11` and `StatusInfected.TimePointd33`.

Over to you: linear models

1. Read `samplesheet_corrected.tsv`, build `~ Status` and find the reference level
2. Use `fct_relevel()` to make Uninfected the reference
3. Fit `~ Status + TimePoint`: how many coefficients? What is the reference group?
4. Spot the interactions in the five panels

DISCUSS

One gene has log2FoldChange 2.8 (about 7×) but p = 0.36. Another has 1.9 (about 3.7×) and p = 0.0003.

How can the bigger change be less significant?

Hint: compare their `lfcSE` and `stat` columns.

`~ Status` · `~ Status + TimePoint` · `~ Status + TimePoint + Status:TimePoint`

Differential expression with DESeq2

DESeq2 needs three ingredients:

1

Raw counts

The txi object saved after Salmon and tximport

`txi.rds`

2

Sample metadata

The corrected sample sheet, with Uninfected as the reference

`samplesheet_corrected.tsv`

3

A design formula

Which factors we expect to affect expression

`~ Status`

DESeq2 does **not** check that samples are in the same order in the counts and the metadata — that's your job.

We'll build up three models

MODEL 1

Simple

```
~ Status
```

Infected vs uninfected, ignoring time point. Used to learn the DESeq2 workflow.

MODEL 2

Additive

```
~ TimePoint + Status
```

Both factors, assuming the infection effect is the same at day 11 and day 33.

MODEL 3

Interaction

```
~ TimePoint + Status
  + TimePoint:Status
```

Lets the effect of infection differ between time points.

Each step answers a more specific biological question — and the PCA will tell us which model fits our data best.

Building the DESeqDataSet

```
txi <- readRDS("RObjects/txi.rds")
sampleinfo <- read_tsv("data/samplesheet_corrected.tsv", col_types = "cccc")
all(colnames(txi$counts) == sampleinfo$SampleName)   # must be TRUE

sampleinfo <- mutate(sampleinfo, Status = fct_relevel(Status, "Uninfected"))

ddsObj.raw <- DESeqDataSetFromTximport(txi = txi,
                                       colData = sampleinfo,
                                       design = ~ Status)

keep <- rowSums(counts(ddsObj.raw)) > 5
ddsObj.filt <- ddsObj.raw[keep, ]
```

AVERAGE TRANSCRIPT LENGTH

tximport also recorded each gene's average transcript length per sample. DESeq2 folds this into its normalisation.

SAME FILTER AS BEFORE

We drop genes with 5 or fewer reads in total, leaving the same 20,091 genes we explored.

What DESeq() does

STEP 1

`estimateSizeFactors()`

Normalise for library size (and transcript length) with the median-ratio method

STEP 2

`estimateDispersions()`

Estimate how variable each gene is between replicates

STEP 3

`nbinomWaldTest()`

Fit the negative binomial model to every gene and compute Wald statistics

```
ddsObj <- DESeq(ddsObj.filt)
```

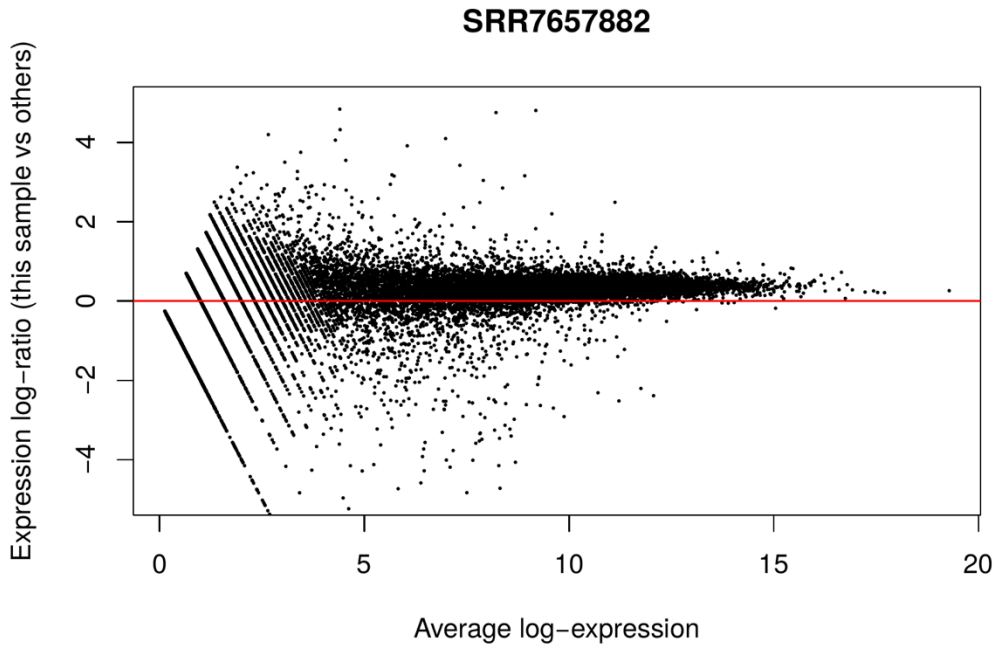
One wrapper runs all three steps. Run them separately only when you want to change a step's defaults.

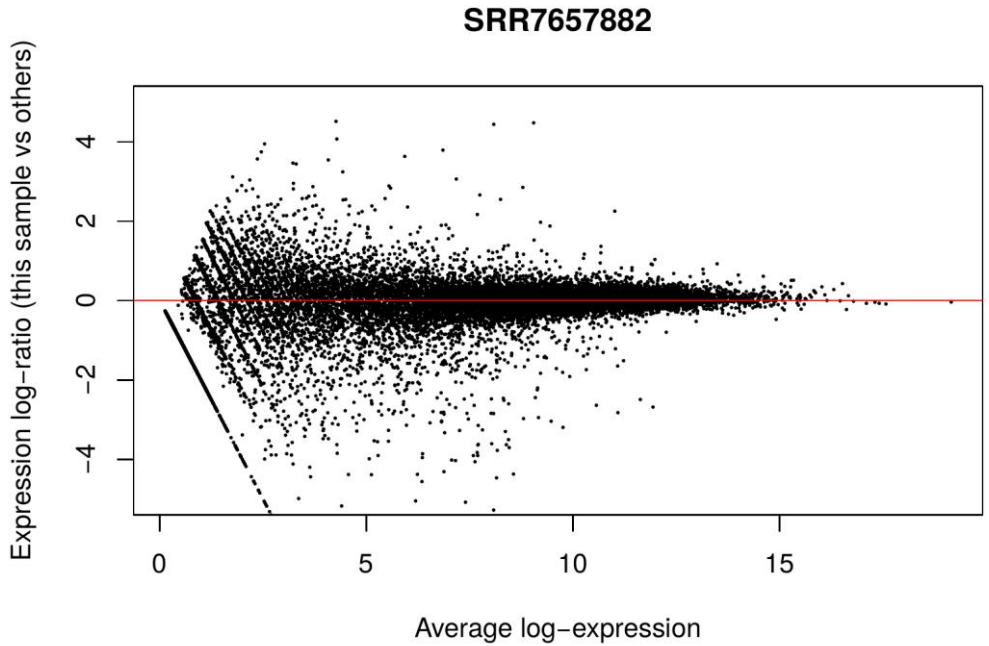
Step 1: size factors

MA plots: compares one sample, SRR7657882, with the average of the other 11 samples:

M = this sample's log2(count + 1) − the others' average
i.e. $\log_2(a) - \log_2(b) = \log_2(a \div b)$

A = the average of the two





BEFORE: RAW LOG COUNTS

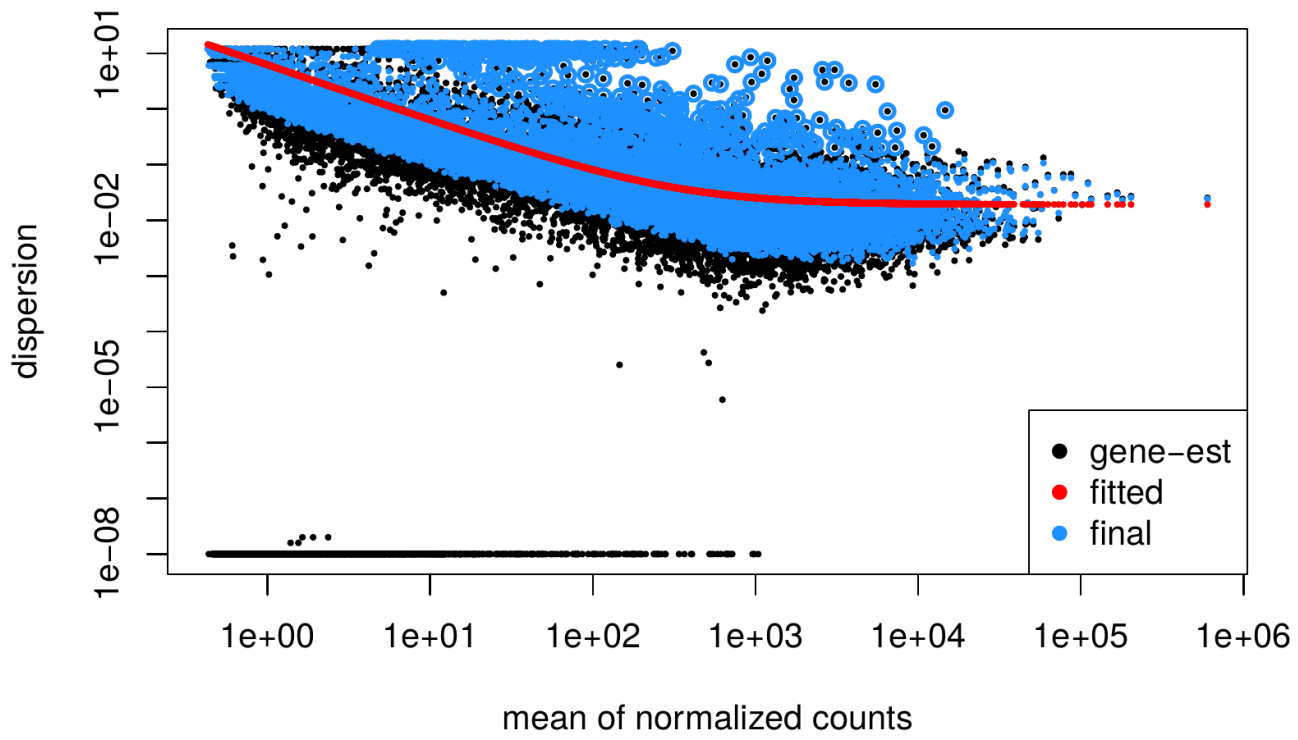
The cloud sits above zero: this sample simply has more reads than the others.

AFTER: NORMALISED COUNTS

Centred on zero: most genes are not changing, as we expect.

DESeq2 never changes your counts: the size factors enter the model as an **offset**.

Step 2: dispersion estimates



Dispersion measures how much a gene varies between replicates.

Gene-wise

Each gene on its own: noisy with only 3 replicates

Fitted

The trend across all genes

Final

Each gene pulled towards the trend

Borrowing information across genes gives more reliable estimates from few replicates.

Always check this plot if you change the defaults.

Getting a results table

```
results.simple <- results(ddsObj, alpha = 0.05)
results.simple

# log2 fold change (MLE): Status Infected vs Uninfected
# Wald test p-value: Status Infected vs Uninfected
# DataFrame with 20091 rows and 6 columns
```

CHECK THE HEADER

It tells you which comparison you are looking at. A positive log2FoldChange means higher in Infected.

INDEPENDENT FILTERING

DESeq2 drops genes too lowly expressed to ever be significant; their padj is `NA`.

Set `alpha` to your FDR cut-off — the default is 0.1.

The additive model

```
additive.model <- as.formula(~ TimePoint + Status)
# ... rebuild ddsObj.raw with this design, filter, then:
ddsObj <- DESeq(ddsObj.filt)

resultsNames(ddsObj)
results(ddsObj, name = "TimePoint_d33_vs_d11",
        alpha = 0.05)
```

Coefficient	β	Meaning
`Intercept`	β_0	Uninfected, day 11
`TimePoint_d33_vs_d11`	β_1	Day 33 vs day 11
`Status_Infected_vs_Uninfected`	β_2	Infected vs uninfected

THE DEFAULT CONTRAST

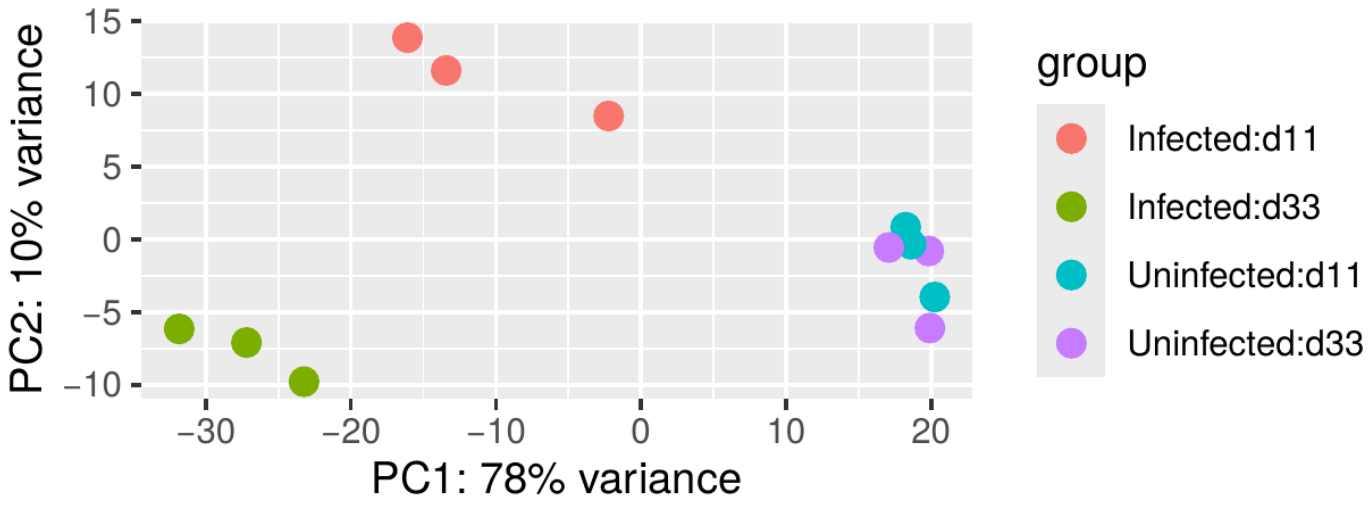
`results()` with no arguments returns the **last column of the model matrix**.

Here that's Status — so the order of terms in your formula matters.

BEST PRACTICE

Always ask for the comparison you want with `name =` or `contrast =`.

Do we need an interaction?



Infected: d11 and d33 separate

Time changes expression in infected mice.

Uninfected: d11 and d33 overlap

Time has little effect without infection.

So the infection effect depends on time

That is an interaction — so we fit one.

```
vstcounts <- vst(ddsObj.raw, blind = TRUE);  plotPCA(vstcounts, intgroup = c("Status", "TimePoint"))
```

Infection effect at each time point

```
design = ~ TimePoint + Status + TimePoint:Status
```

INFECTED VS UNINFECTED · DAY 11

1,072 genes at padj < 0.05

```
name = "Status_Infected_vs_Uninfected"
```

The main effect of Status is infected vs uninfected at the reference time point, day 11.

INFECTED VS UNINFECTED · DAY 33

2,782 genes at padj < 0.05

```
contrast = list(c(
  "Status_Infected_vs_Uninfected",
  "TimePointd33.StatusInfected"))
```

Add the interaction term to get the infection effect at day 33.

Infection changes far more genes in the chronic phase than in the acute phase.

Over to you: DESeq2

1. Simple model: how many genes are significantly **up** and **down** at FDR < 0.05?
2. Additive model: how many coefficients? What is the default contrast?
3. Extract d33 vs d11 with `name =` and count the DE genes
4. Interaction model: extract d33 vs d11 for infected and for uninfected mice
5. Save `ddsObj.interaction` and both results objects

Hint: ?results — see the examples for name and contrast

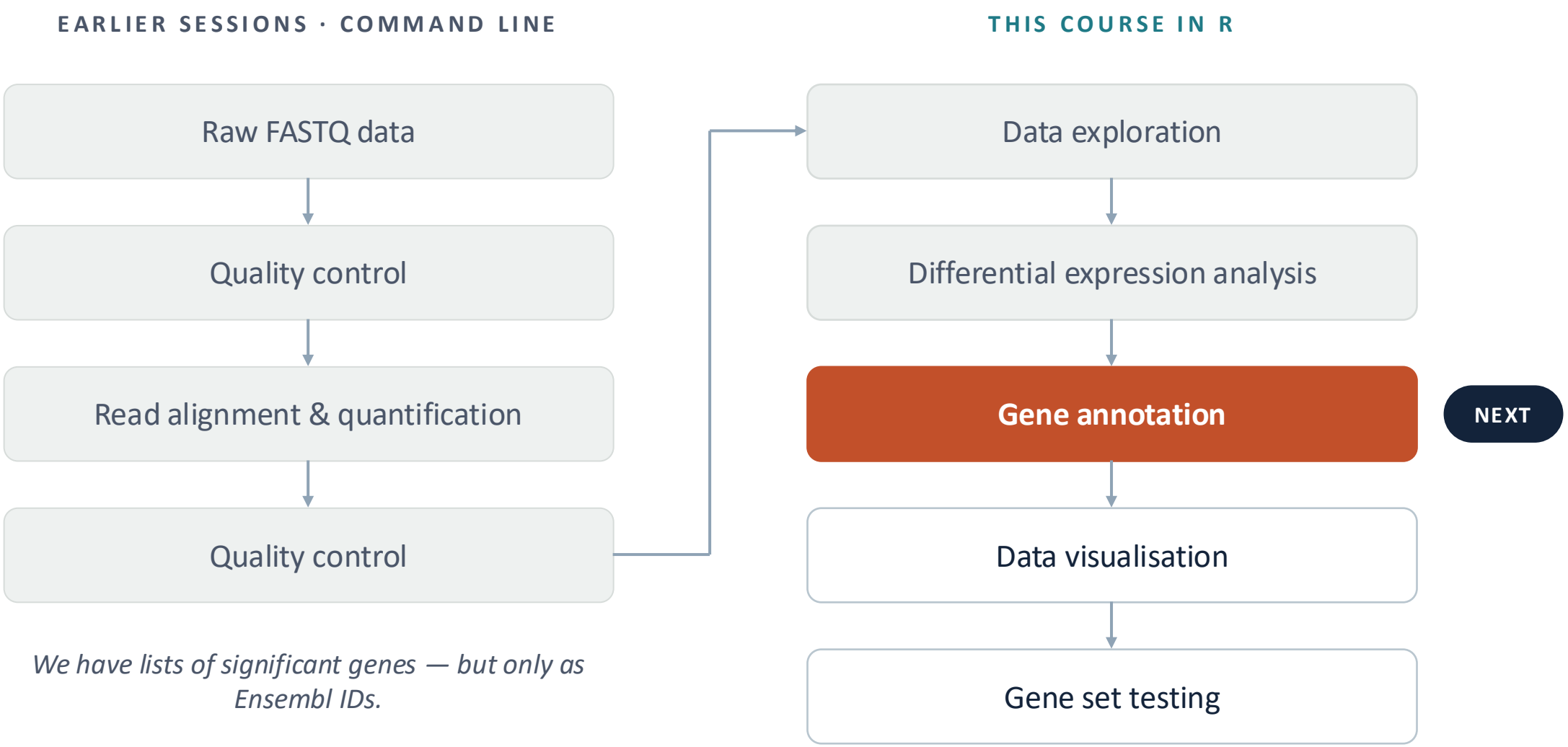
DISCUSS

A gene with log2FoldChange −2.46 has padj 0.27; one with −1.61 has padj 0.02. Why?

How many genes change between d33 and d11 in uninfected mice? Is that surprising?

Do these results suggest another way to analyse this dataset?

Next: gene annotation



We have lists of significant genes — but only as Ensembl IDs.

From an ID to a gene

`ENSMUSG00000026064`

What DESeq2 gives us



`gene_name`	**Ptp4a1** The gene symbol people recognise
`description`	**protein tyrosine phosphatase 4a1** What the gene is
`entrezid`	**19243** NCBI Entrez ID — needed for gene set testing

The rule that matters most: **use the same annotation release as your reference.**
Our counts came from Ensembl release 102 (GRCm38), so our annotation must too.

Four ways to get annotation

Bioconductor packages

Installed like any package; rebuilt periodically with the latest annotation.

biomaRt

Queries the online BioMart service; lets you match the exact Ensembl version. Extended materials.

TODAY

AnnotationHub

Online database of annotation resources, kept fairly up to date and cached on your machine after first use.

Download a table

e.g. GENCODE tables mapping Ensembl IDs to symbols and Entrez IDs. Simple, but less flexible.

All four can give the same answer. What matters is knowing which release you used and recording it.

Finding our database in AnnotationHub

STEP 1

Connect

```
ah <-
AnnotationHub()
```

Over 71,000 records; cached locally

STEP 2

Search

```
query(ah, c("EnsDb",
  "Mus musculus",
"102"))
```

Returns one record: AH89211

STEP 3

Download

```
ah[[dbResult$ah_id]]
```

[ ] describes · [[ ]] downloads

STEP 4

Extract genes

```
genes(MouseEnsDb,
  return.type =
"data.frame")
```

One row per gene

Field	AH89211
`title`	Ensembl 102 EnsDb for Mus musculus
`genome`	GRCm38
`rdataclass`	EnsDb

Check the record before you download: **right species, right genome build, right Ensembl release.**

Keeping the columns we need

```
annot <- annotations %>%
    dplyr::select(gene_id, gene_name, description, entrezid) %>%
    dplyr::filter(gene_id %in% rownames(results.d11))
```

WHY dplyr::select?

Annotation packages have their own `select()`. If they load after the tidyverse, they hide dplyr's version and you get a confusing "unable to find an inherited method" error.

Fix: write `dplyr::select()`, or load tidyverse last.

WHAT WE KEEP

`gene_id` to join onto our results

`gene_name` the gene symbol

`description` what the gene is

`entrezid` for gene set testing

IDs don’t map one-to-one

20,091
genes in our results

2,782
have no Entrez ID

17,278
unique Entrez values (including NA)

SOME ENSEMBL IDS SHARE AN ENTREZ ID

gene_id	gene_name	entrezid
ENSMUSG00000026064	Ptp4a1	**19243**
ENSMUSG00000117310	Ptp4a1	**19243**
ENSMUSG00000115958	Gm28040	**280287**
ENSMUSG00000116158	Kiss1	**280287**

RESOLVING DUPLICATES

- Check them by hand online
- Keep the one with the most counts
- Remove one or all of them

There’s no perfect answer — it’s your decision. For the course we load a pre-resolved table.

Annotating the results

```
# pre-resolved annotation table
annot <- readRDS("RObjects/Ensembl_annotations.rds")

results.d11 <- readRDS(
    "RObjects/DESeqResults.interaction_d11.rds")

results.d11 <- as.data.frame(results.d11) %>%
    rownames_to_column("GeneID") %>%
    left_join(annot, "GeneID")

write_tsv(results.d11, "results.d11_Results_Annotated.txt")
```

`rownames_to_column()`

Turns the Ensembl IDs from row names into a GeneID column we can join on

`left_join()`

Keeps every DE result, adding annotation where it exists — missing annotation becomes NA

`write_tsv()`

A readable table you can open in Excel or share

Over to you: annotation

1. Connect to AnnotationHub and find the Ensembl 102 mouse EnsDb
2. Download it and extract the gene table with `genes()`
3. Count genes with missing and duplicated Entrez IDs
4. Load the resolved annotation, join it to `results.d11` and save
5. Repeat for the day 33 results

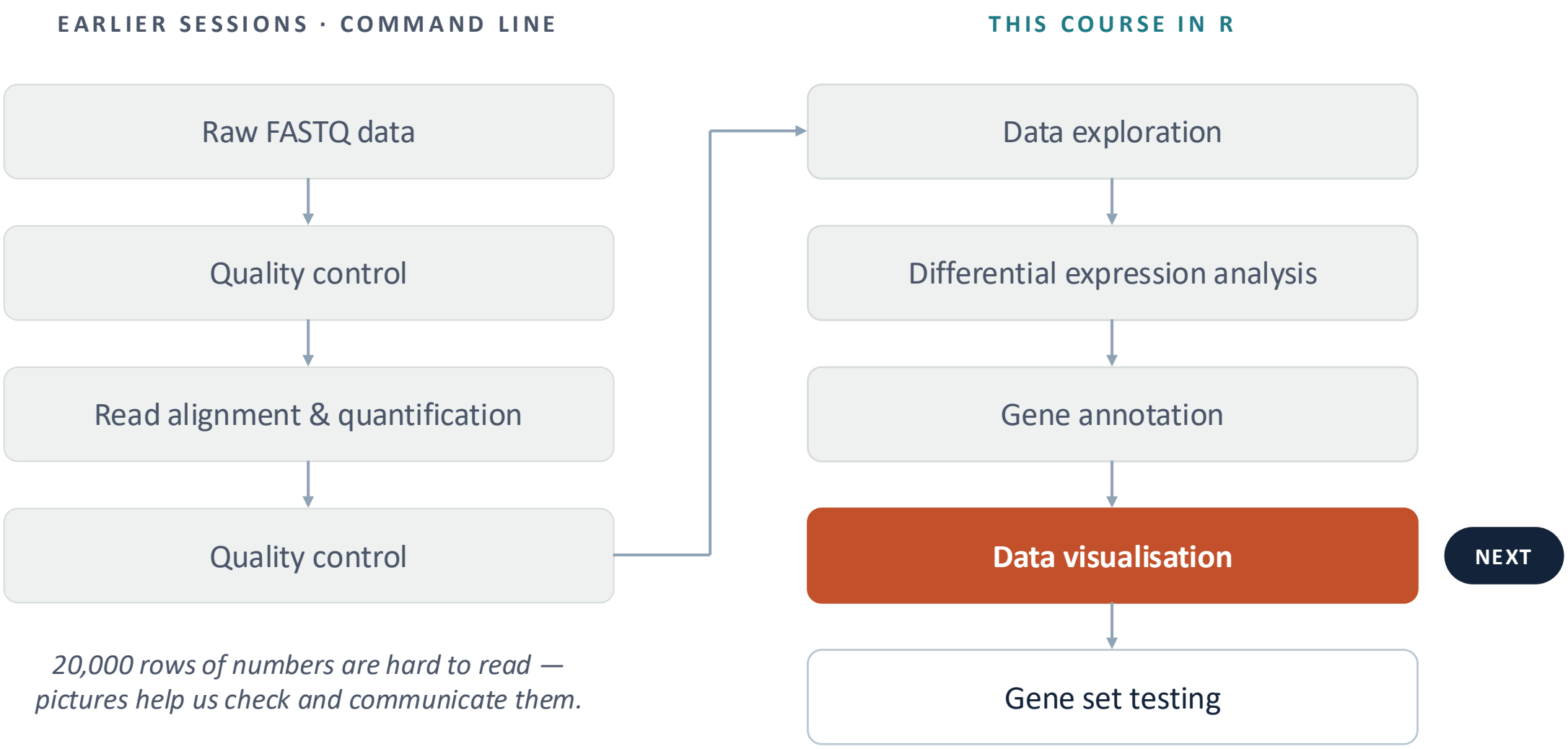
If select() fails: use dplyr::select()

DISCUSS

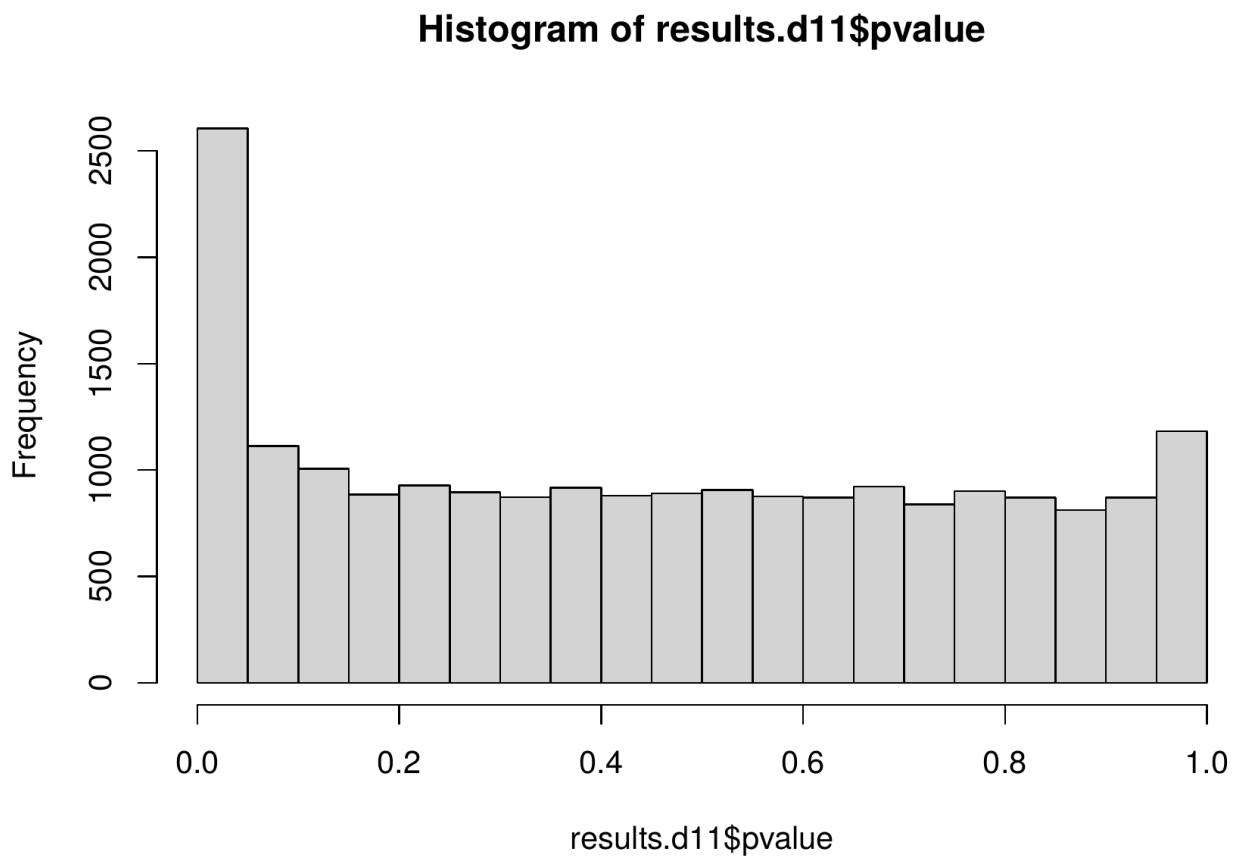
What could go wrong if you annotated with a newer Ensembl release than the one you quantified against?

Two Ensembl IDs share an Entrez ID. Which would you keep, and why?

Next: data visualisation



A first sanity check: p-value histogram



WHAT A HEALTHY ONE LOOKS LIKE

A spike near 0

Genes that are genuinely changing

A flat tail

Unchanged genes: spread evenly from 0 to 1

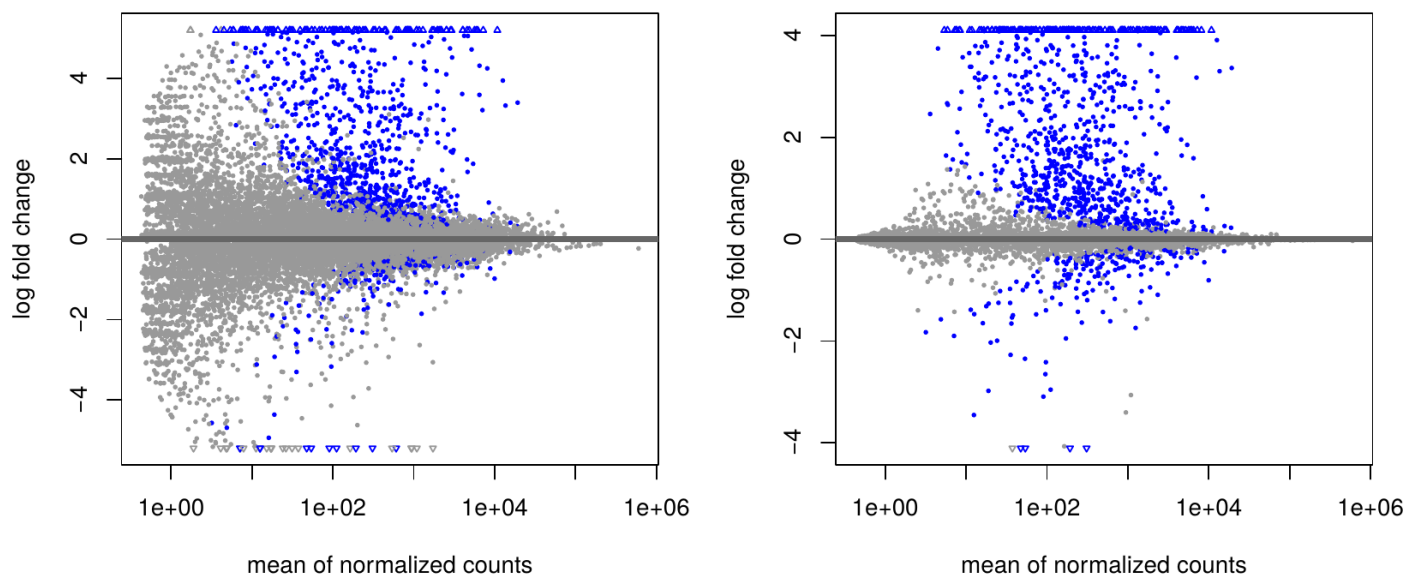
Other shapes — a hump in the middle, or a slope — are a warning that the model may not fit the data well.

```
hist(results.d11$pvalue)
```

Shrinking log2 fold changes

Low-count genes have noisy fold changes, so they can look more dramatic than well-measured genes.

`lfcShrink()` pulls unreliable estimates towards zero using an **empirical Bayes approach.**



Before **After lfcShrink()**

```
shrink.11 <- lfcShrink(
  ddsObj,
  res = results.d11,
  type = "ashr")
```

Pass `res =` so your `padj` values keep alpha = 0.05.

Use shrunk values for plotting and ranking genes; significance is unchanged.

Volcano plots with ggplot2

```
ggplot(shrinkTab.11,
       aes(x = log2FoldChange,
           y = -log10(pvalue))) +
  geom_point(aes(colour = padj < 0.05)) +
  geom_text(data = ~top_n(.x, 1, wt = -padj),
            aes(label = Symbol))
```

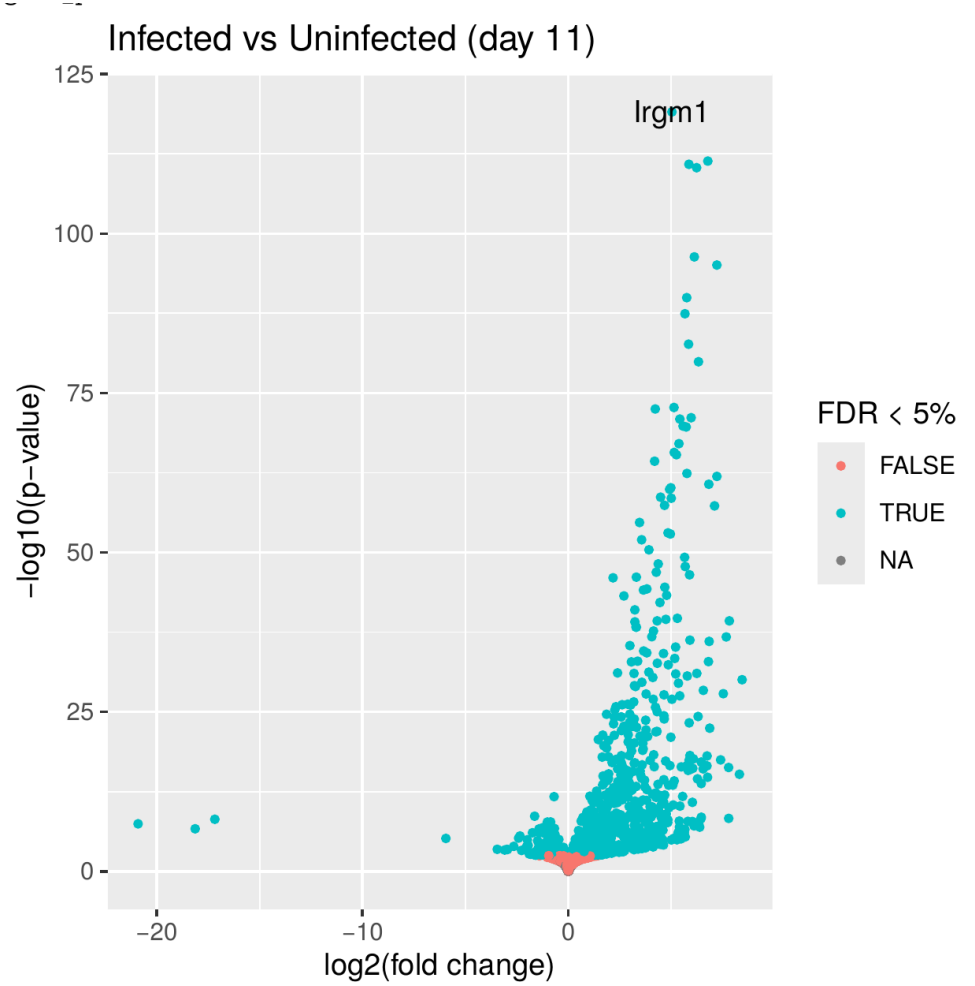
`data` the shrunk, annotated results table

`aes()` maps columns to x, y and colour

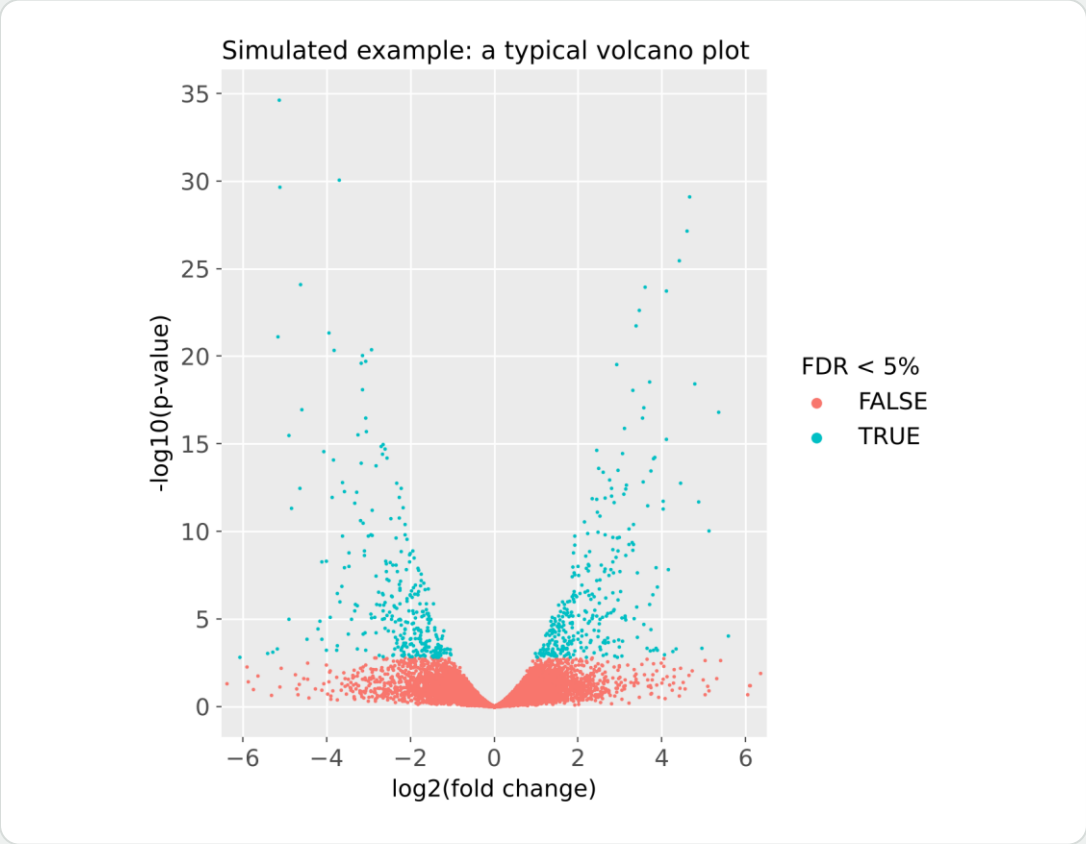
`geom_point()` draws one point per gene

`geom_text()` labels the most significant gene

Fold change across, significance up: the most interesting genes sit high and to the sides.

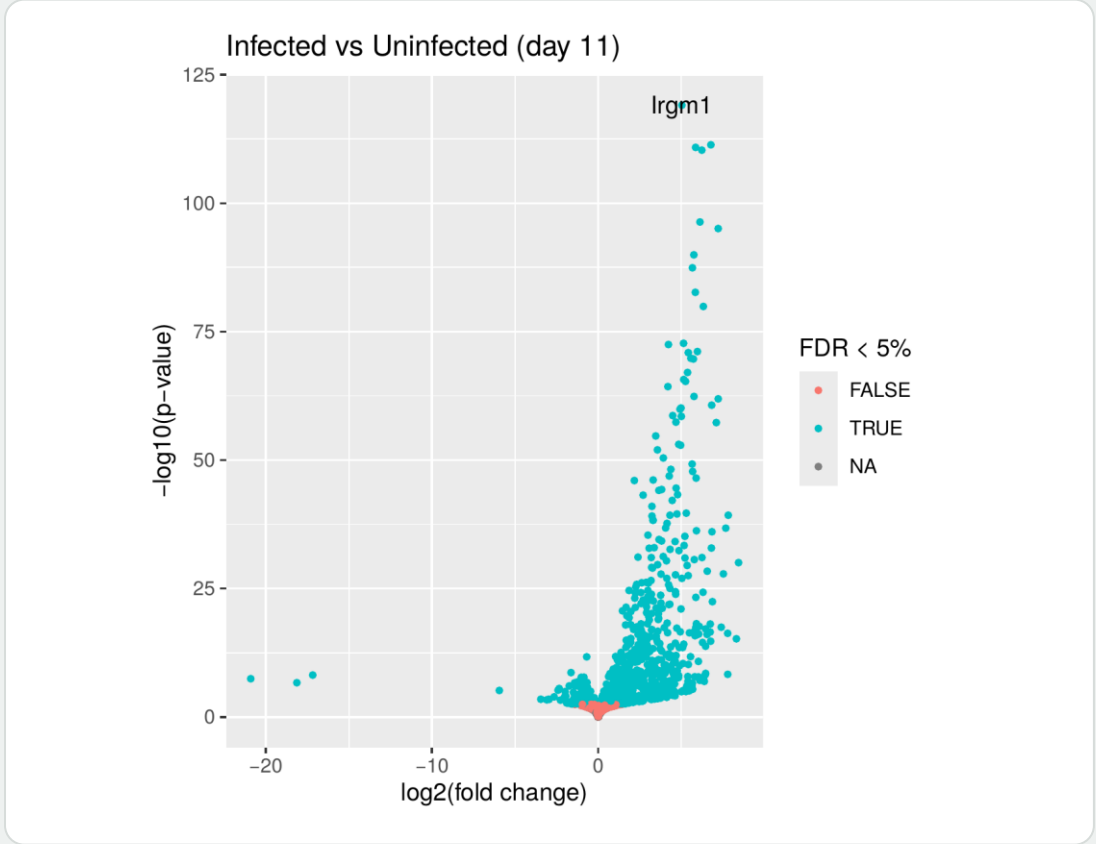


What a volcano plot usually looks like



TYPICAL (SIMULATED DATA)

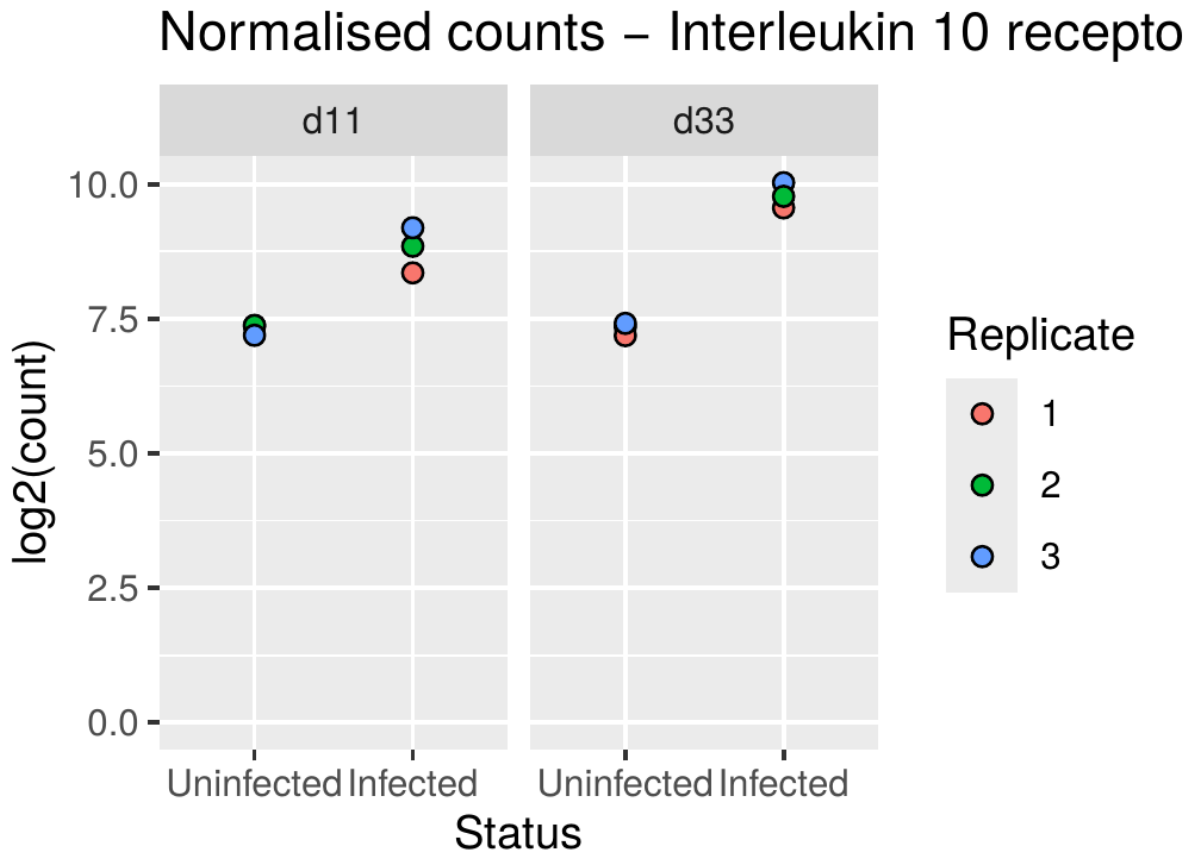
Roughly symmetrical: similar numbers of genes go up and down.



OUR DATA: INFECTED VS UNINFECTED, DAY 11

Lopsided: infection mainly switches genes on (878 up, 194 down).

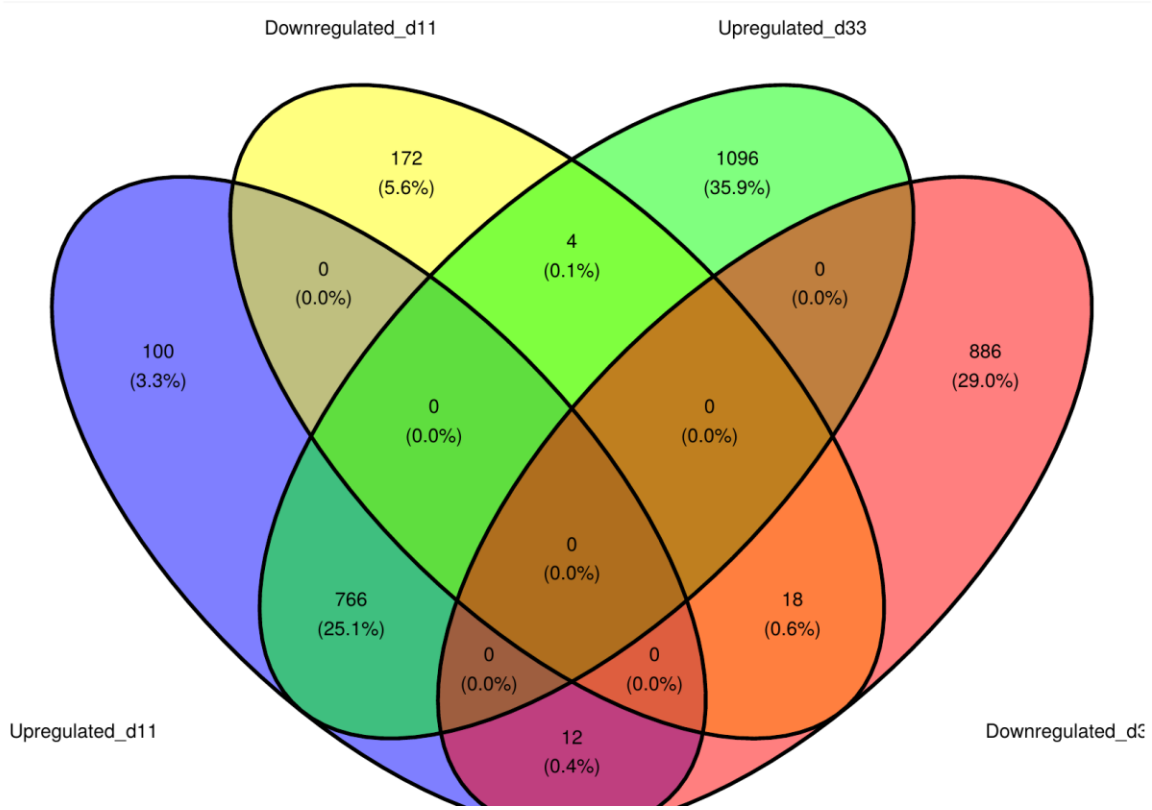
Look at the samples behind a result



Before following a gene up in the lab, check every sample. ***Il10ra*** is higher in infected mice at both time points, and the replicates agree.

```
plotCounts(ddsObj, gene = geneID,
  intgroup = c("TimePoint", "Status",
               "Replicate"),
  returnData = TRUE) %>%
  ggplot(aes(x = Status,
             y = log2(count))) +
  geom_point(aes(fill = Replicate),
             shape = 21, size = 2) +
  facet_wrap(~TimePoint)
```

Comparing day 11 and day 33



878

up at day 11

194

down at day 11

1,866

up at day 33

916

down at day 33

766 genes are up-regulated at both time points; very few change direction.

```
vennList <- list(
  Upregulated_d11 =
    getGenes(shrinkTab.11, "up"), ...)
ggvenn(vennList)
```

Heatmap of the top 300 genes

1. **Choose genes**

   Top 300 by padj at day 11

2. **Transform**

   vst() counts, so values are comparable

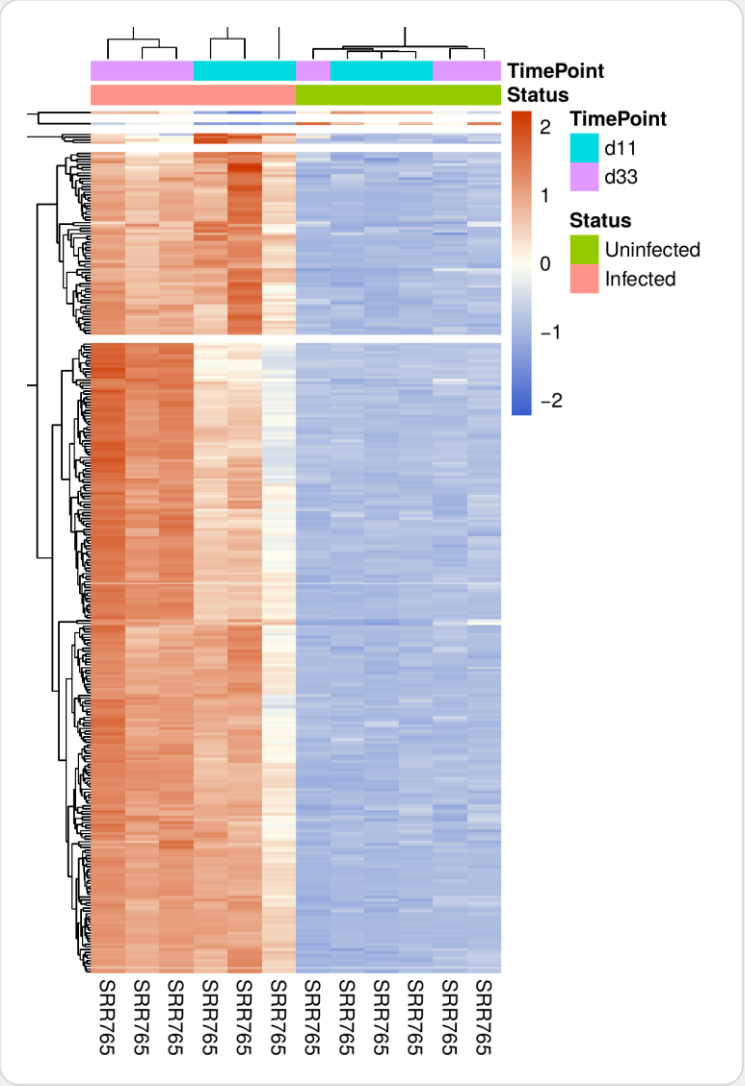
3. **Scale each row**

   scale = "row": z-scores, so each gene is shown relative to its own mean

4. **Cluster and annotate**

   Genes and samples are clustered; coloured bars show Status and TimePoint

```
pheatmap(plotDat, scale = "row",
  color = hmPalette, cutree_rows = 5,
  annotation_col = annot_df)
```



Over to you: visualisation

1. Shrink and annotate the **day 33** results; draw its volcano plot
2. Make a day 33 MA plot in ggplot2: `log2(baseMean)` vs `log2FoldChange`
3. Draw a strip chart for *Jchain*
4. Build the Venn diagram and the heatmap

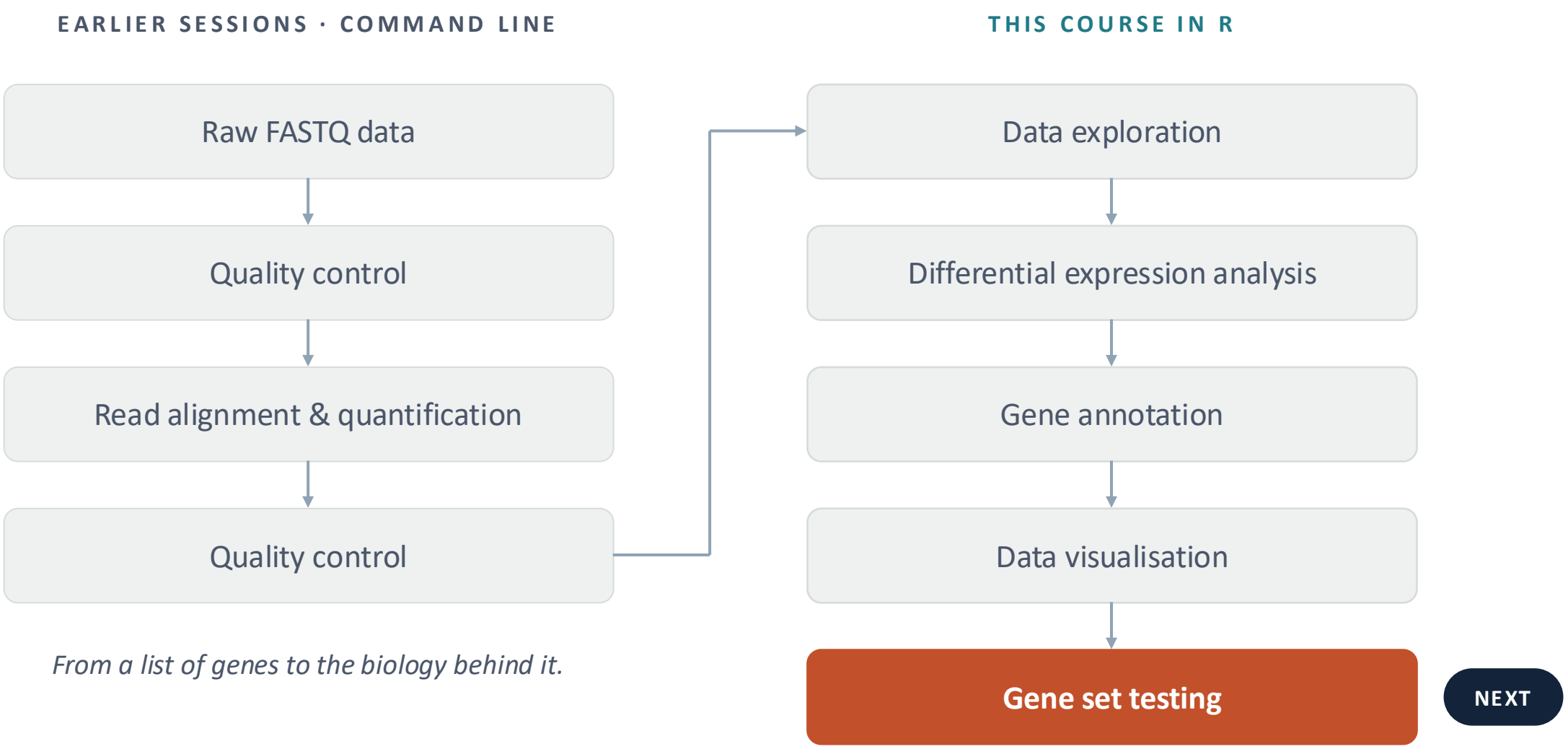
DISCUSS

Compare the day 11 and day 33 volcano plots. What differences do you see?

Which plot would you put in a paper, and which would you use only to check your analysis?

Save the shrunk tables: you'll need them for gene set testing.

Last step: gene set testing



From a list of genes to the biology behind it.

From genes to pathways

A **gene set** is a group of genes that share something: a pathway, a biological function, a chromosomal location. We ask whether our results are enriched for any of them.

OVER-REPRESENTATION (ORA)

Uses a list of significant genes

- Pick genes passing a cut-off
- Are pathway genes over-represented in the list?
- KEGG and GO, with clusterProfiler

GENE SET ENRICHMENT (GSEA)

Uses every gene, ranked

- Rank all genes, e.g. by fold change
- Do a set's genes cluster at the top or bottom?
- MSigDB Hallmark sets, with clusterProfiler

A cut-off is arbitrary: GSEA can pick up many small, coordinated changes that ORA would miss.

Over-representation: a 2 × 2 table

	Differentially expressed	Not DE
In pathway	**a** DE and in the pathway	**b**
Not in pathway	**c**	**d**

Fisher's exact test: is **a** bigger than we'd expect if pathway membership and DE status were independent?

READING THE OUTPUT

Pathway: Antigen processing and presentation

`GeneRatio` **39/344**

39 of our 344 significant genes are in the pathway

`BgRatio` **87/9467**

87 of 9,467 annotated genes are in the pathway

11% of our list vs under 1% of the background: strongly enriched.

KEGG pathway enrichment

```
search_kegg_organism("mouse", by = "common_name")  # mmu

sigGenes <- shrink.d11 %>%
    drop_na(Entrez, padj) %>%
    filter(padj < 0.05 & abs(log2FoldChange) > 1) %>%
    pull(Entrez)

keggRes <- enrichKEGG(gene = sigGenes, organism = "mmu")
```

Significant (`padj < 0.05`) and at least 2-fold (`|LFC| > 1`), with an Entrez ID — KEGG needs Entrez IDs.

`browseKEGG()` opens a pathway online; `pathview()` saves a pathway figure coloured by fold change.

TOP HITS AT DAY 11 (74 PATHWAYS)

Pathway	Genes
Antigen processing and presentation	**39**
Epstein–Barr virus infection	**55**
Graft-versus-host disease	**31**
Type I diabetes mellitus	**32**
Phagosome	**47**

Disease names come from KEGG's categories — they point to shared immune machinery, not those diseases.

Gene Ontology enrichment

GO terms come in three ontologies:
biological process, **molecular function** and **cellular component**.

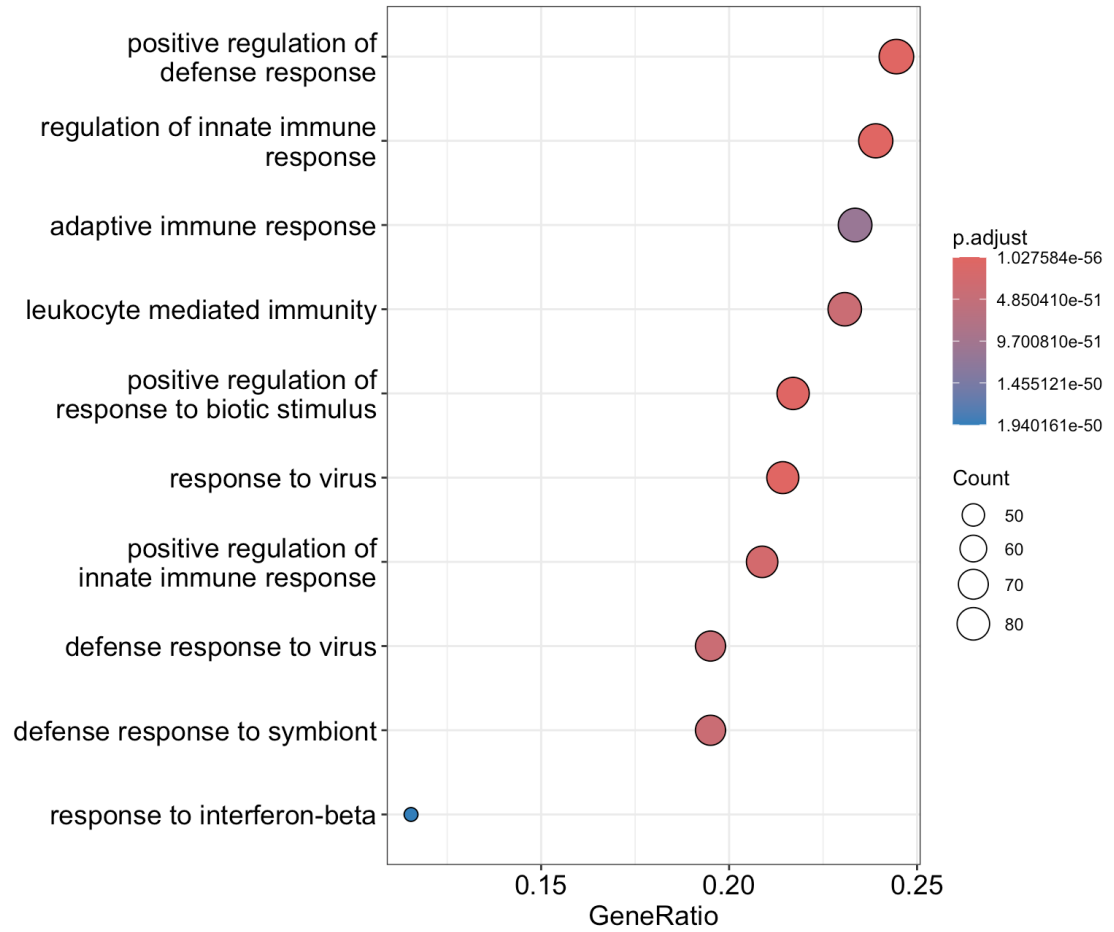
We test biological processes.

```
ego <- enrichGO(
  gene = sigGenes_GO,
  universe = universe,
  OrgDb = org.Mm.eg.db,
  keyType = "ENSEMBL",
  ont = "BP",
  readable = TRUE)

dotplot(ego)
```

`universe` = every gene we tested: the right background.

dotplot – Top GO terms



GeneRatio is the fraction of your significant genes that belong to that GO term: GeneRatio = Count ÷ (number of your significant genes that have any GO annotation)

GSEA: does a gene set sit at one end?

1 Rank every gene

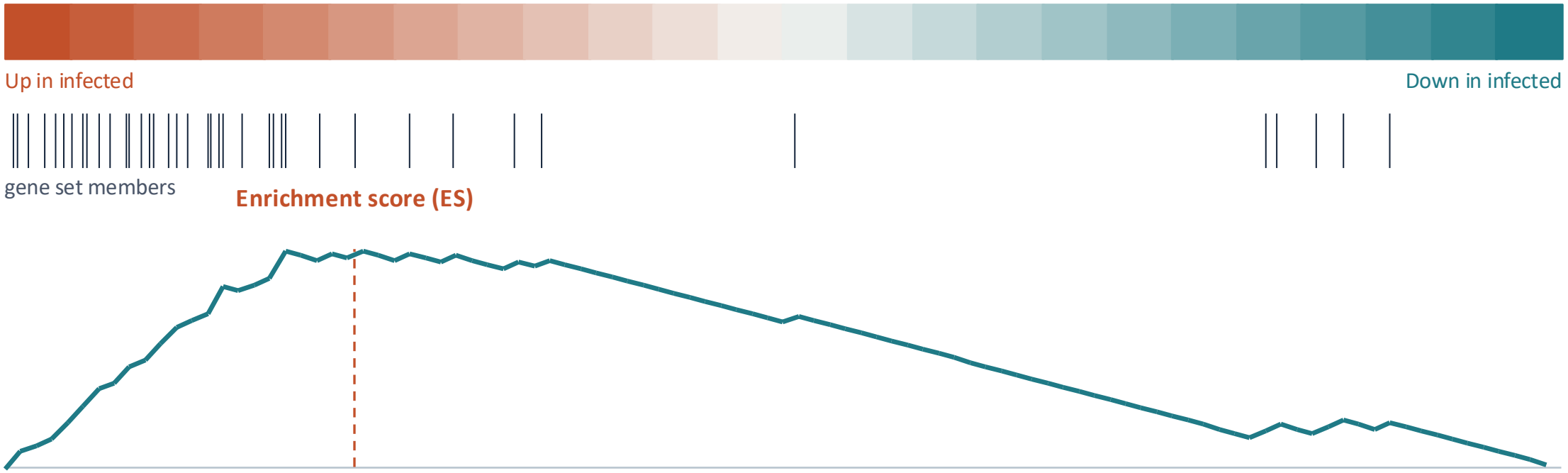
by a metric such as the shrunk log2 fold change

2 Mark the gene set

one tick for each member in the ranking

3 Walk down the list

score goes up at each member, down otherwise; the peak is the enrichment score



A set spread evenly through the list would stay near zero.

NES – ES normalized for the size of the gene set as small sets can reach high scores by chance more easily

Running GSEA on the Hallmark gene sets - MSigDB

```
msigdb.mm <- getMsigdb(org = "mm", id = "EZID",
                       version = "2023.1")
hallmarks <- subsetCollection(msigdb.mm, "h")

rankedGenes <- shrink.d11 %>%
  drop_na(GeneID, padj, log2FoldChange) %>%
  filter(!is.na(Entrez)) %>%
  arrange(desc(log2FoldChange)) %>%
  pull(log2FoldChange, Entrez)

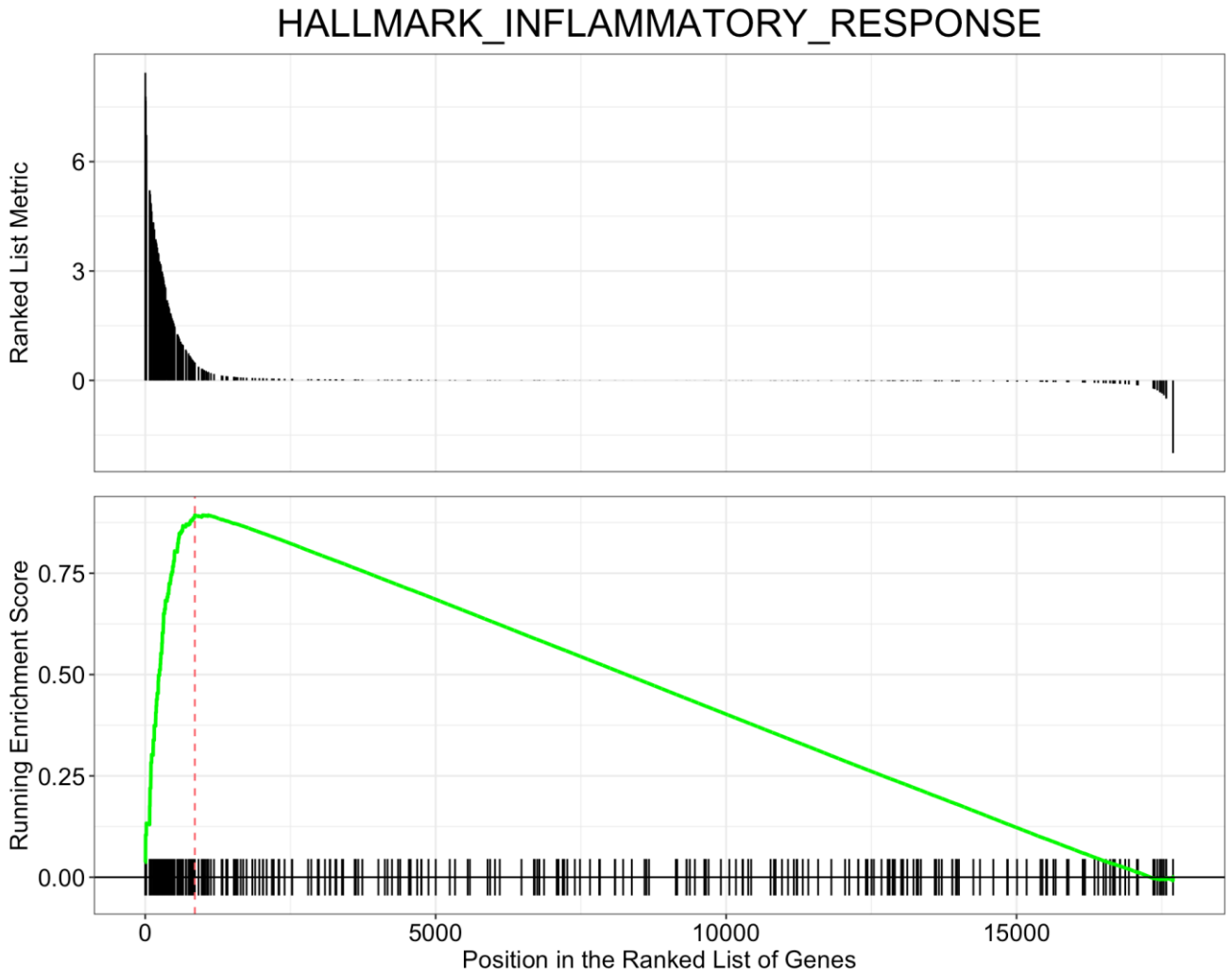
gseaRes <- GSEA(rankedGenes, TERM2GENE = term2gene,
                minGSSize = 15, maxGSSize = 500)
```

TOP HALLMARK SETS AT DAY 11

Gene set	NES
Interferon alpha response	1.40
Interferon gamma response	1.39
Allograft rejection	1.36
IL6–JAK–STAT3 signalling	1.36
Inflammatory response	1.31

All positive: these sets sit among genes switched on by infection. Interferon-γ is central to the immune response to Toxoplasma.

Reading an enrichment plot



Top panel

The ranking metric: shrunk log2 fold change, highest on the left

Black ticks

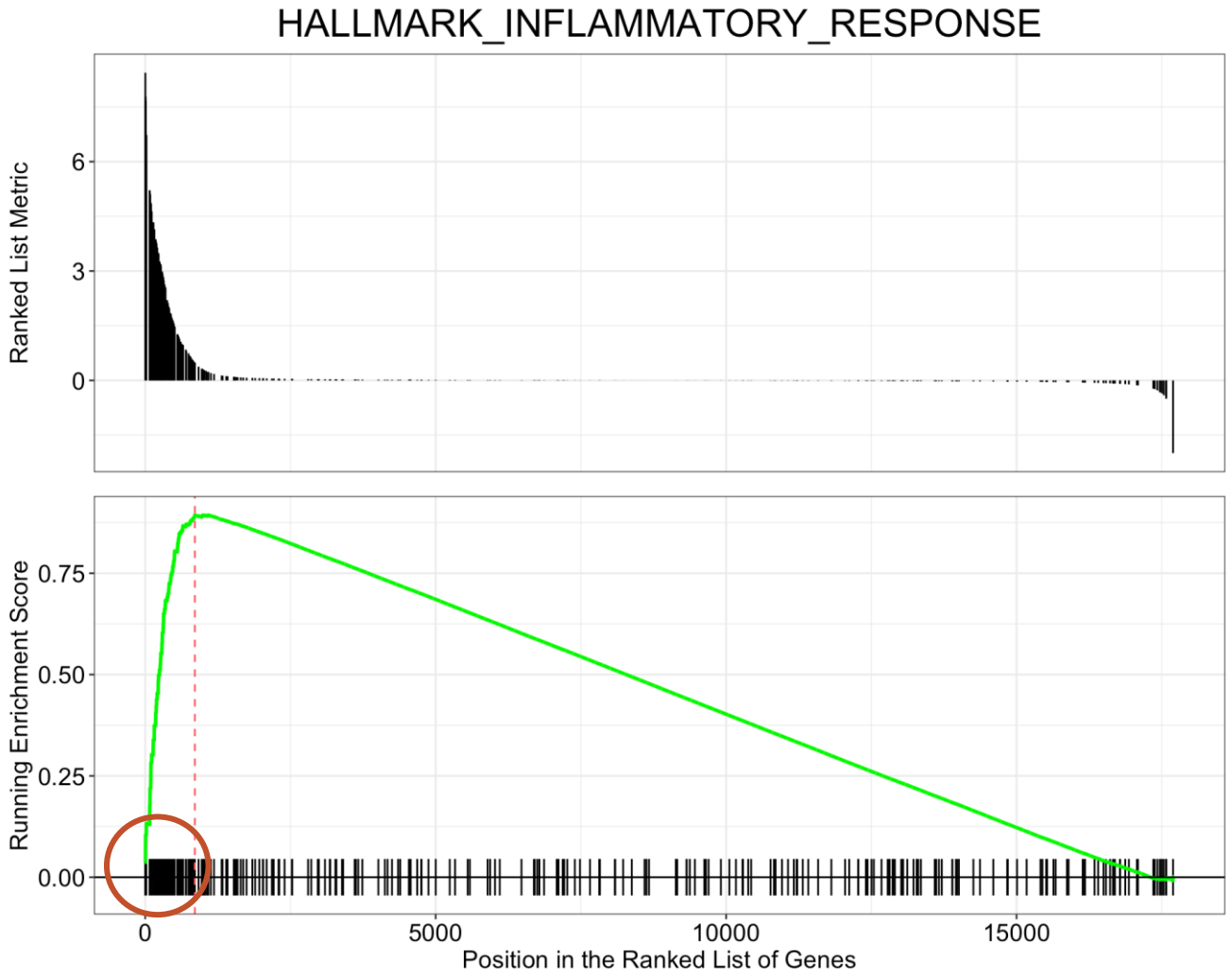
Genes in the Inflammatory Response set — bunched on the left

Green curve

Running enrichment score; its early peak is the ES

```
gseaplot(gseaRes,
  geneSetID =
"HALLMARK_INFLAMMATORY_RESPONSE")
```

Reading an enrichment plot



Top panel

The ranking metric: shrunk log2 fold change, highest on the left

Black ticks

Genes in the Inflammatory Response set — bunched on the left

Green curve

Running enrichment score; its early peak is the ES

```
gseaplot(gseaRes,
  geneSetID =
"HALLMARK_INFLAMMATORY_RESPONSE")
```

Leading edge genes – they're the core of the signal – often the most interesting genes to follow up on

Over to you: gene set testing

1. Run `enrichKEGG()` and `enrichGO()` for day 11
2. Exercise 1: use `pathview()` for mmu04659 or mmu04658, using only genes with padj < 0.01
3. Run GSEA on the Hallmark sets and draw an enrichment plot
4. Exercise 2: rank by `-log10(pvalue) * sign(log2FoldChange)` and rerun GSEA, then repeat for day 33

DISCUSS

Does ranking by fold change or by signed p-value change your top gene sets? Why might it?

How do the pathways enriched at day 33 differ from day 11?

KEGG and GSEA use Entrez IDs here; GO uses Ensembl IDs